
SOFTWARE REQUIREMENT SPECIFICATION

for

LKFORUM WEBSITE

Version 1.0

Prepared by : Vo An Khoi (23520790)

Submitted to : Nguyen Trinh Dong
Lecturer

May 24, 2026

Contents

1	Introduction	3
1.1	Purpose	3
1.2	Intended Audience and Reading Suggestions	3
1.3	Project Scope	3
1.4	References	3
2	Overall Description	4
2.1	Product Perspective	4
2.2	Operating Environment	4
2.3	Design and Implementation Constraints	5
2.4	Assumptions and Dependencies	5
3	System Features	6
3.1	Use Case Description	6
3.1.1	UC-01 - Sign Up	6
3.1.2	UC-02 - Sign In	9
3.1.3	UC-03 - Forgot Password	15
3.1.4	UC-04 - Manage Profile	19
3.1.5	UC-05 - Change Settings	21
3.1.6	UC-06 - View Activity History	24
3.1.7	UC-07 - Follow / Unfollow User	26
3.1.8	UC-08 - Browse Post	27
3.1.9	UC-09 - Search Content	30
3.1.10	UC-10 - Create Post	32
3.1.11	UC-11 - Edit Post	36
3.1.12	UC-12 - Delete Post	38
3.1.13	UC-13 - Post Comment	41
3.1.14	UC-14 - Reply to Comment	43
3.1.15	UC-15 - Vote on Content	45
3.1.16	UC-16 - Participate in Poll	48
3.1.17	UC-17 - Save Post	51
3.1.18	UC-18 - Hide Post	53
3.1.19	UC-19 - Manage Drafts	56
3.1.20	UC-20 - Private Messaging	58
3.1.21	UC-21 - Manage Notifications	62
3.1.22	UC-22 - Report Violation	64
3.1.23	UC-23 - Create Community	67
3.1.24	UC-24 - Approve / Reject Post	69
3.1.25	UC-25 - Ban / Mute User	72
3.1.26	UC-26 - Configure Community	75
3.1.27	UC-27 - Assign Roles	78
3.1.28	UC-28 - Handle Global Reports	81
3.1.29	UC-29 - Manage Platform	84

3.1.30 UC-30 - View Analytics	87
3.2 Data Requirements	90
3.2.1 Logical Data Model	90
3.2.2 Data Dictionary	91
3.3 Data Integrity, Retention, and Disposal	109
4 Other Nonfunctional Requirements	110
4.1 Performance Requirements	110
4.2 Security Requirements	110
4.3 Software Quality Attributes	110
5 Other Requirements	111
6 Appendix	112
6.1 Messages Codes	112
6.2 Error Codes	112

1 Introduction

1.1 Purpose

This SRS describes the functional and nonfunctional requirements for software release 1.0 of the Community Social Network Website (LKForum). This document is intended to be used by the members of the project team who will implement and verify the correct functioning of the system. Unless otherwise noted, all requirements specified here are committed for release 1.0

1.2 Intended Audience and Reading Suggestions

This SRS is for developers, project managers, users and testers. Further the discussion will provide all the internal, external, functional and also non-functional informations about "LKFORUM WEBSITE".

1.3 Project Scope

This SRS describes the functional and nonfunctional requirements for software release 1.0 of the Community Social Network Website (LKForum). This document is intended to be used by the members of the project team who will implement and verify the correct functioning of the system. Unless otherwise noted, all requirements specified here are committed for release 1.0

1.4 References

2 Overall Description

2.1 Product Perspective

Community Social Network LKForum is a new website that provides a comprehensive, modern solution for building and managing online communities. The context diagram in Figure 1 illustrates the external entities and system interfaces for release 1.0. The system is expected to evolve over several releases, ultimately connecting to third-party social media APIs for authentication (Google, Facebook) and expanding to include advanced content monetization tools and mobile-native applications.

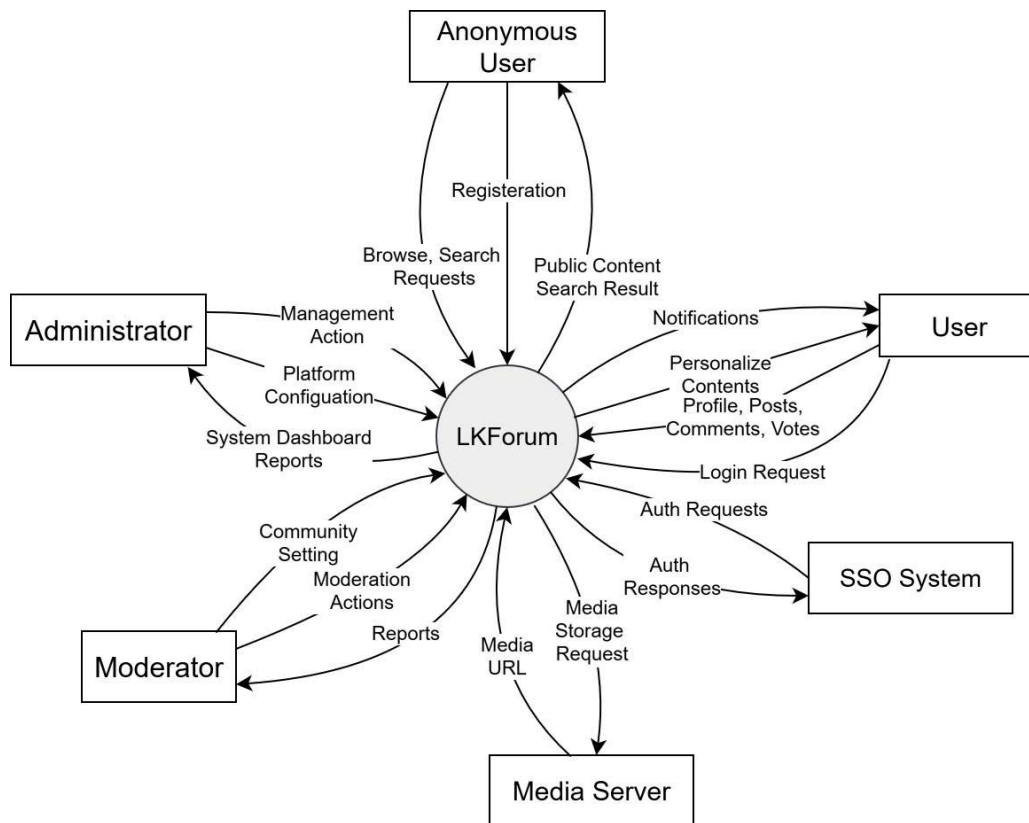


Figure 2.1: Context diagram for release 1.0 of LKForum

2.2 Operating Environment

OE-1: LKForum must operate correctly on modern web browsers including: Google Chrome (latest version), Mozilla Firefox (latest version), Apple Safari (latest version), and Microsoft Edge (latest version).

OE-2: Users can access LKForum via the Internet from desktop computers, laptops, and tablets. The interface must have responsive design to be compatible with various screen sizes.

2.3 Design and Implementation Constraints

- CO-1:** The user interface (front-end) source code must be developed using the Svelte framework.
- CO-2:** The system must use a standard MongoDB NoSQL database with a back-end developed using the Go framework.
- CO-3:** All HTML code must adhere to the HTML 5.0 standard.

2.4 Assumptions and Dependencies

- AS-1:** It is assumed that users have a stable Internet connection to access and use the platform's features.
- DE-1:** The social media login feature (Google, Facebook) depends on the APIs and policies of these third-party service providers. Any changes on their part may affect the operation of this feature.

3 System Features

The LKForum website is being built using the Go programming language for the back-end, Svelte with Vite for the front-end, MongoDB for primary data storage, and Redis for caching and session management.

Back-End – Go with the Gin web framework.

Front-End – Svelte with Vite.

Database – MongoDB.

Cache and Session Store – Redis.

3.1 Use Case Description

3.1.1 UC-01 - Sign Up

Name	Sign Up
Description	This use case describes how a new user creates an account and verifies the email address using OTP.
Actor	User
Trigger	When the user clicks the “Sign Up” button.
Pre-condition	The user is not logged in to the system. The user is on the sign up page.
Post-condition	The user account is created and verified. The user is authenticated and can access the system.

Table 3.1: Use Case Specification - Sign Up

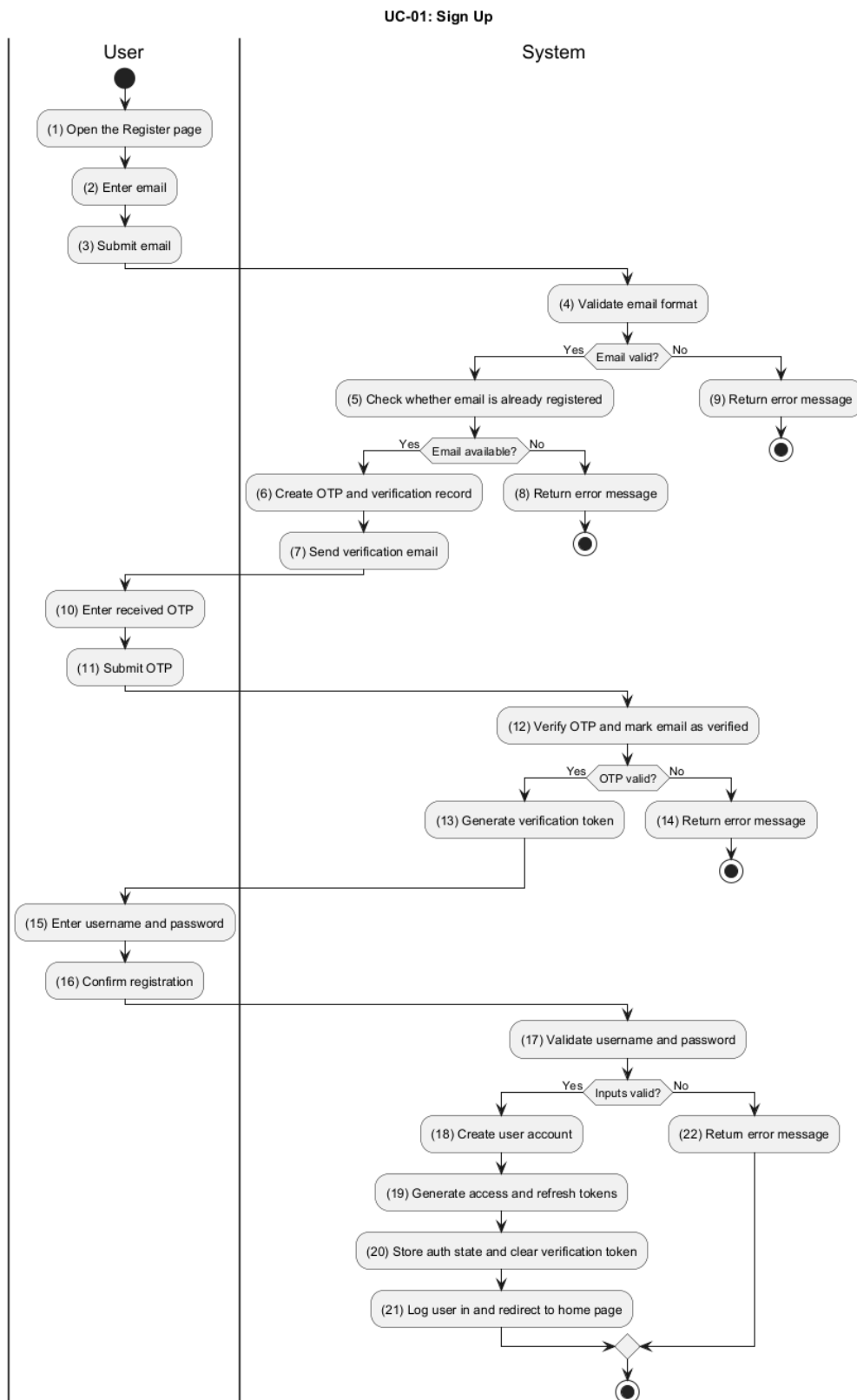


Figure 3.1: Activity Diagram for UC-01 Sign Up

ID	Activ- ity	Rule Type	Description
BR-UC01-01	(4)	Validation Rules	Validate Email Format: - The system checks that email is present and valid. - If the email is missing or invalid, the system returns a bad-request error.
BR-UC01-02	(5)	Constraint Rules	Check Email Availability and Prepare Verification Record: - The system checks whether the email already exists in the user repository. - If the email already exists, the system returns an email-exists error.
BR-UC01-03	(7)	Message Rules	Send Verification Email: If an email sender is configured, the system sends the OTP by email asynchronously.
BR-UC01-04	(12)	Validation Rules	Verify OTP and Issue Token: - The system checks that email and otp are present. - If no verification record exists, the system returns an invalid OTP error. - If the verification record is already marked verified, the system returns an email-already-verified error. - If the OTP does not match or has expired, the system returns an invalid or expired OTP error. - Otherwise, the system marks the verification record as verified and generates a verification token from the verified email and nonce.
BR-UC01-05	(17)	Validation Rules	Validate Registration Fields: - The system checks that username is between 3 and 20 characters. - The system checks that password is at least 8 characters long and strong enough.

Table 3.2: Business Rules for UC-01 - Sign Up

ID	Activ-ity	Rule Type	Description
BR-UC01-06	(18–20)	Workflow Rules	Create Account and Complete Login: - The system hashes the password with <i>GenerateFromPassword()</i> before storing it. - The system checks that the username is still available and that the email is still not registered. - The system creates the new local user with default settings, deletes the verification record, invalidates the username cache, and generates access and refresh tokens using <i>GenerateToken()</i>.
BR-UC01-07	(21)	Redirect Rules	Redirect After Registration: The system logs the user in and redirects to the home page.

Table 3.2: Business Rules for UC-01 - Sign Up

3.1.2 UC-02 - Sign In

Name	Sign In
Description	This use case describes how a registered user logs into the system using local credentials or Google OAuth.
Actor	User
Trigger	When the user clicks the “Sign In” button.
Pre-condition	The user is not logged in to the system. The user is on the sign in page.
Post-condition	The user is authenticated. The user is redirected to the appropriate page.

Table 3.3: Use Case Specification - Sign In

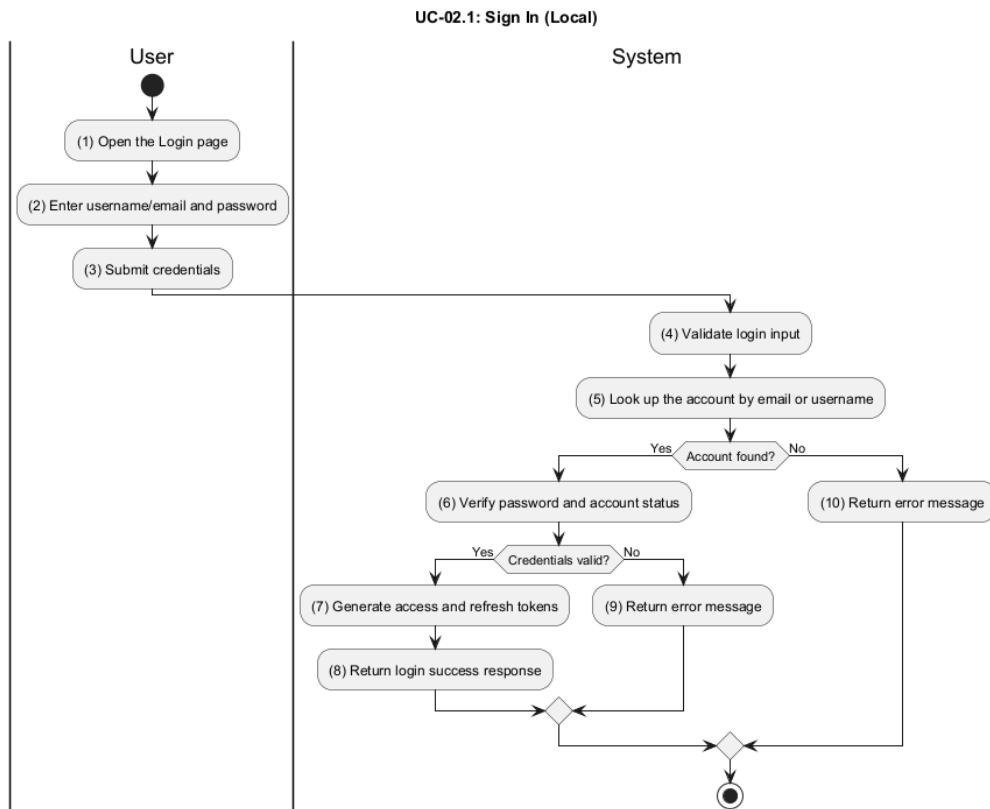


Figure 3.2: Activity Diagram for UC-02 Local Sign In

ID	Activ-ity	Rule Type	Description
BR-UC02-01	(4)	Validation Rules	Validate Login Input: - The controller checks the JSON body for identifier and password in UserLoginRequest. - If binding fails or either field is missing, the controller returns a bad request response. - The controller does not validate the identifier format here; the service decides whether to search by email or username.

Table 3.4: Business Rules for UC-02.1 - Sign In (Local)

ID	Activ- ity	Rule Type	Description
BR-UC02-02	(5)	Workflow Rules	Look Up Account: - The service checks whether identifier is an email using <i>isEmail()</i>. - The service looks up the account by email or username using the user repository. - If no account exists, the service returns an invalid credentials error.
BR-UC02-03	(6)	Authorization Rules	Verify Password and Account Status: - The system checks that the account uses the local provider. - The service compares the stored password with password using <i>bcrypt.CompareHashAndPassword()</i>. - If the password hash is empty or the comparison fails, the service returns an invalid credentials error. - If IsVerified is false, the service returns an email-not-verified error. - If the user is banned and the ban has not expired, the service returns a user-inactive error.
BR-UC02-04	(7–10)	Message Rules	Return Authentication Response: - The system returns an error message for invalid credentials. - The system returns the authenticated user data together with the access and refresh tokens on success.

Table 3.4: Business Rules for UC-02.1 - Sign In (Local)

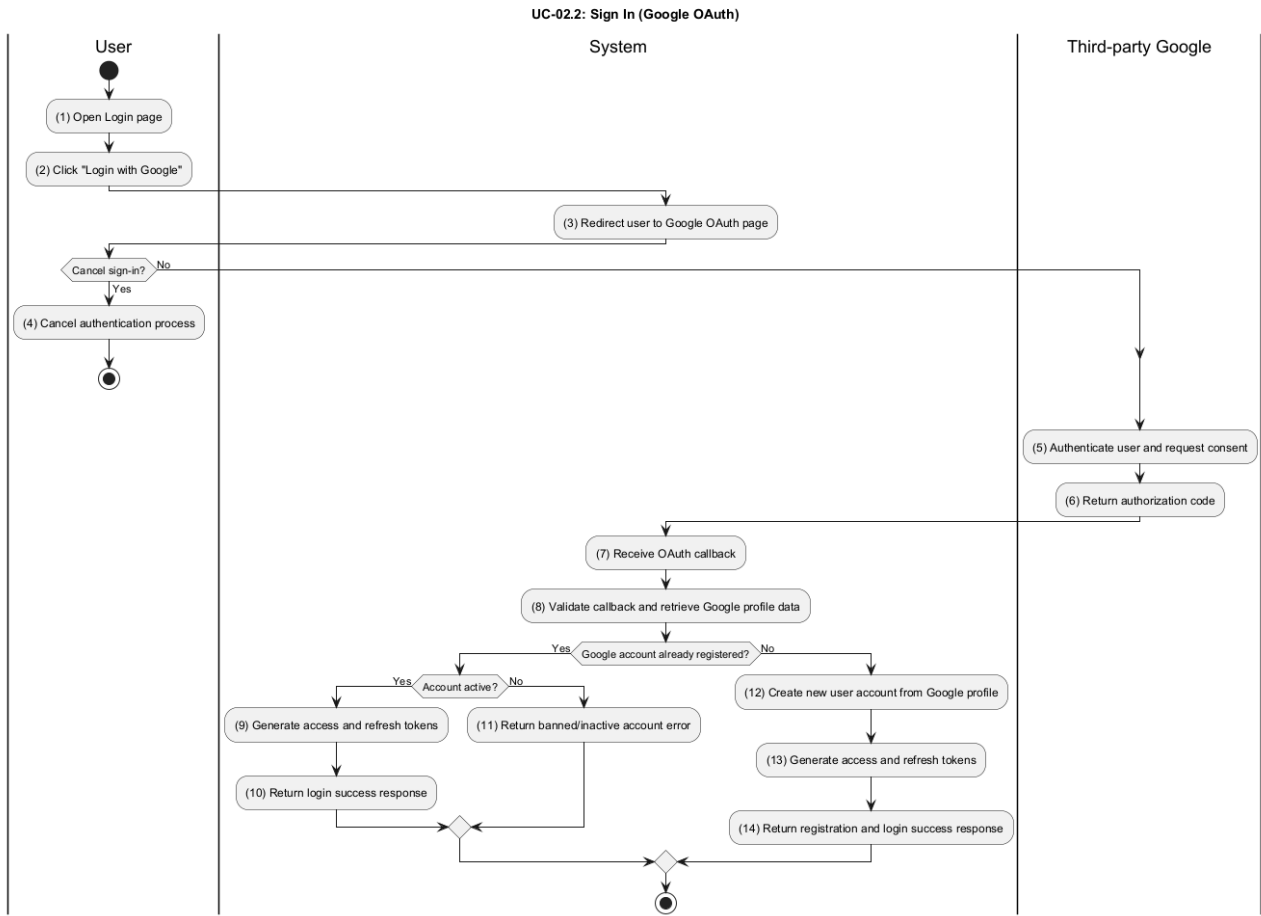


Figure 3.3: Activity Diagram for UC-02 Google Sign In

ID	Activ-ity	Rule Type	Description
BR-UC02-05	(3)	Redirect Rules	Start Google OAuth Flow: - The system redirects the user to Google OAuth using a generated state value. - Google authenticates the user and returns an authorization code.

Table 3.5: Business Rules for UC-02.2 - Sign In (Google OAuth)

ID	Activ- ity	Rule Type	Description
BR-UC02-06	(8)	Validation Rules	Process Google Callback: - The controller checks that the callback contains code. - If code is missing, the controller redirects to the frontend error page with a missing-auth-code message. - The service exchanges code for Google profile data using <i>GetGoogleUserInfo()</i>. - If Google rejects the authorization code or profile lookup fails, the controller redirects to the frontend error page with the returned error message.
BR-UC02-07	(9–11)	Authorization Rules	Login or Setup Decision: - If an account exists and provider is Google, the service generates access and refresh tokens using <i>GenerateToken()</i>. - If the account belongs to a different login method, the service returns a login-method-mismatch error. - If the account is banned and the ban has not expired, the service returns a user-inactive error. - If no Google account exists, the service creates a setup token using <i>CreateSetupToken()</i> and returns the setup-required result.

Table 3.5: Business Rules for UC-02.2 - Sign In (Google OAuth) (Continued)

ID	Activ- ity	Rule Type	Description
BR-UC02-08	(12)	Validation Rules	Validate Google Setup Input: - The controller checks the JSON body for setup_token and username in CompleteGoogleSetupRequest. - If binding fails or setup_token is missing, the controller returns a bad request response. - The service validates that username is at least 3 characters and no more than 20 characters. - If username already exists, the service returns a username-exists error. - If the Google email is already taken, the service returns an email-exists error.
BR-UC02-09	(13)	Workflow Rules	Create Google Account: - The system parses setup_token to recover the Google claims. - The system creates the user with provider set to Google, stores provider_id, applies default settings, and copies the Google avatar URL when available. - The system generates access and refresh tokens after the user is created.
BR-UC02-10	(14)	Message Rules	Return Setup Success: - The system returns the created user together with the access and refresh tokens. - The system shows the setup-complete success response.

Table 3.5: Business Rules for UC-02.2 - Sign In (Google OAuth)

3.1.3 UC-03 - Forgot Password

Name	Forgot Password
Description	This use case describes how a user resets a forgotten password through email verification and OTP.
Actor	User
Trigger	When the user clicks the “Forgot Password” link.
Pre-condition	The user has an existing account and can access the registered email address. The user is not logged in to the system.
Post-condition	The user password is reset successfully. The user can sign in again with the new password.

Table 3.6: Use Case Specification - Forgot Password

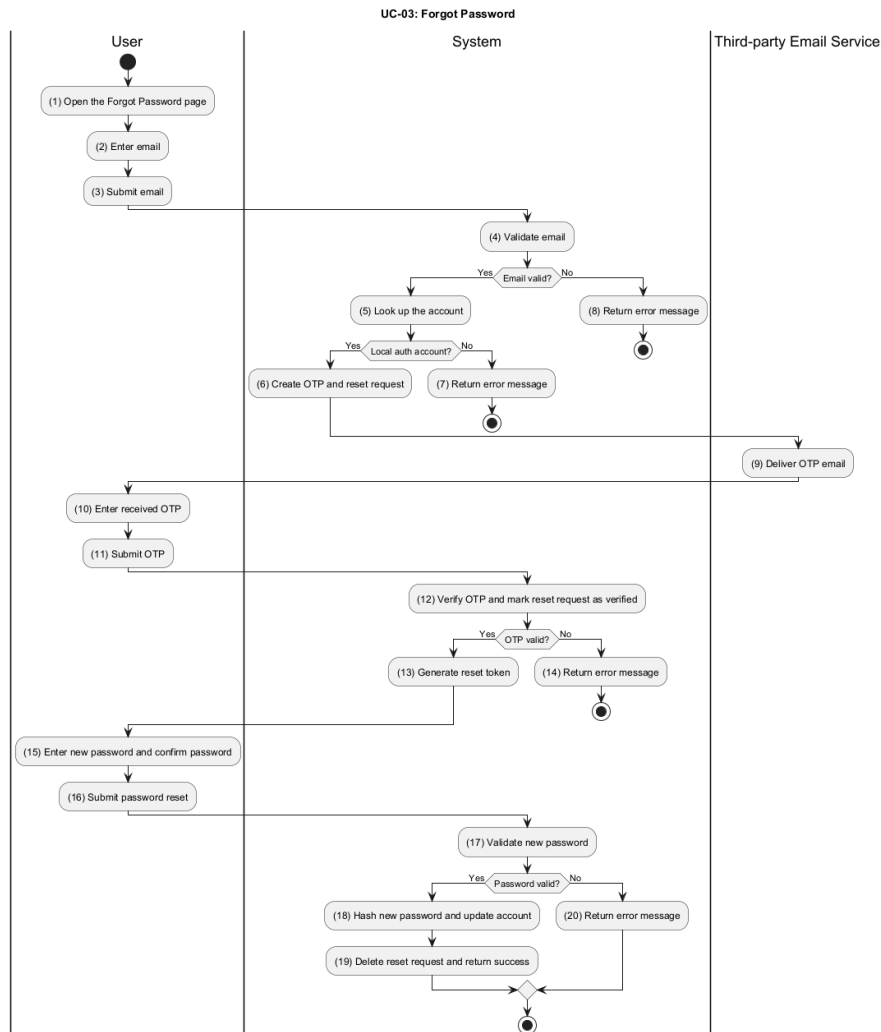


Figure 3.4: Activity Diagram for UC-03

ID	Activ- ity	Rule Type	Description
BR-UC03-01	(4)	Validation Rules	Validate Reset Request: - The controller checks email in <code>ForgotPasswordRequest</code>. - If binding fails or email is missing or invalid, the controller returns a bad request response.
BR-UC03-02	(5)	Authorization Rules	Validate Account Ownership: - The service looks up the account by email using the user repository. - If the account does not exist, the service returns an email-not-registered error. - If the account exists but the provider is not local, the service returns a login-method-mismatch error.
BR-UC03-03	(6)	Workflow Rules	Create Password Reset Request: - The system deletes any existing reset record for the same email. - The system generates a 6-digit OTP, a nonce, and an expiry timestamp using <i><code>generateOTP()</code></i> and <i><code>generateNonce()</code></i>. - The system stores a new password-reset record with IsVerified = false.
BR-UC03-04	(7)	Message Rules	Return Reset Request Result: - If the account is not local, the service returns a login-method-mismatch error. - If the reset record is created successfully, the system sends the reset OTP asynchronously by email. - The system returns a success message telling the user to check the inbox.

Table 3.7: Business Rules for UC-03 - Forgot Password

ID	Activ- ity	Rule Type	Description
BR-UC03-05	(12)	Validation Rules	Validate Reset OTP: - The controller checks email and otp in <code>VerifyResetPasswordOTPRequest</code>. - If binding fails, the controller returns a bad request response. - If no reset record exists for the email, the service returns an invalid OTP error. - If the reset record is already verified, the service returns an email-already-verified error. - If the OTP does not match or has expired, the service returns an invalid or expired OTP error.
BR-UC03-06	(12)	Workflow Rules	Verify OTP and Generate Reset Token: - The service marks the reset record as verified. - The service generates a reset token from the verified email and nonce using <code>CreateVerificationToken()</code>.
BR-UC03-07	(14)	Message Rules	Return OTP Error: If the OTP is invalid or expired, the controller returns the corresponding error message.

Table 3.7: Business Rules for UC-03 - Forgot Password (Continued)

ID	Activ-ity	Rule Type	Description
BR-UC03-08	(17)	Validation Rules	Validate New Password: - The controller checks reset_token and new_password in ResetPasswordRequest. - If binding fails or reset_token is missing, the controller returns a bad request response. - The DTO binding requires new_password to be present and at least 6 characters long. - The service requires new_password to be at least 8 characters long. - The service requires new_password to contain uppercase, lowercase, numeric, and special characters.
BR-UC03-09	(18)	Workflow Rules	Hash and Update Password: - The service parses the reset token and verifies the stored nonce. - The service looks up the user by email. - The service hashes new_password with <i>GenerateFromPassword()</i>. - The service updates the user password in the repository.
BR-UC03-10	(19)	Workflow Rules	Clean Up Reset Record: - The service deletes the reset record after a successful password change. - The controller returns a success response telling the user they can log in with the new password.
BR-UC03-11	(20)	Message Rules	Return Password Error: If the new password is invalid, the controller returns the corresponding error message.

Table 3.7: Business Rules for UC-03 - Forgot Password

3.1.4 UC-04 - Manage Profile

Name	Manage Profile
Description	This use case describes how a user updates profile information such as bio, avatar, and cover image.
Actor	User
Trigger	When the user opens the profile edit page.
Pre-condition	The user is logged in to the system. The user has permission to edit the profile.
Post-condition	The profile information is updated and stored. The new profile data is visible across the system.

Table 3.8: Use Case Specification - Manage Profile

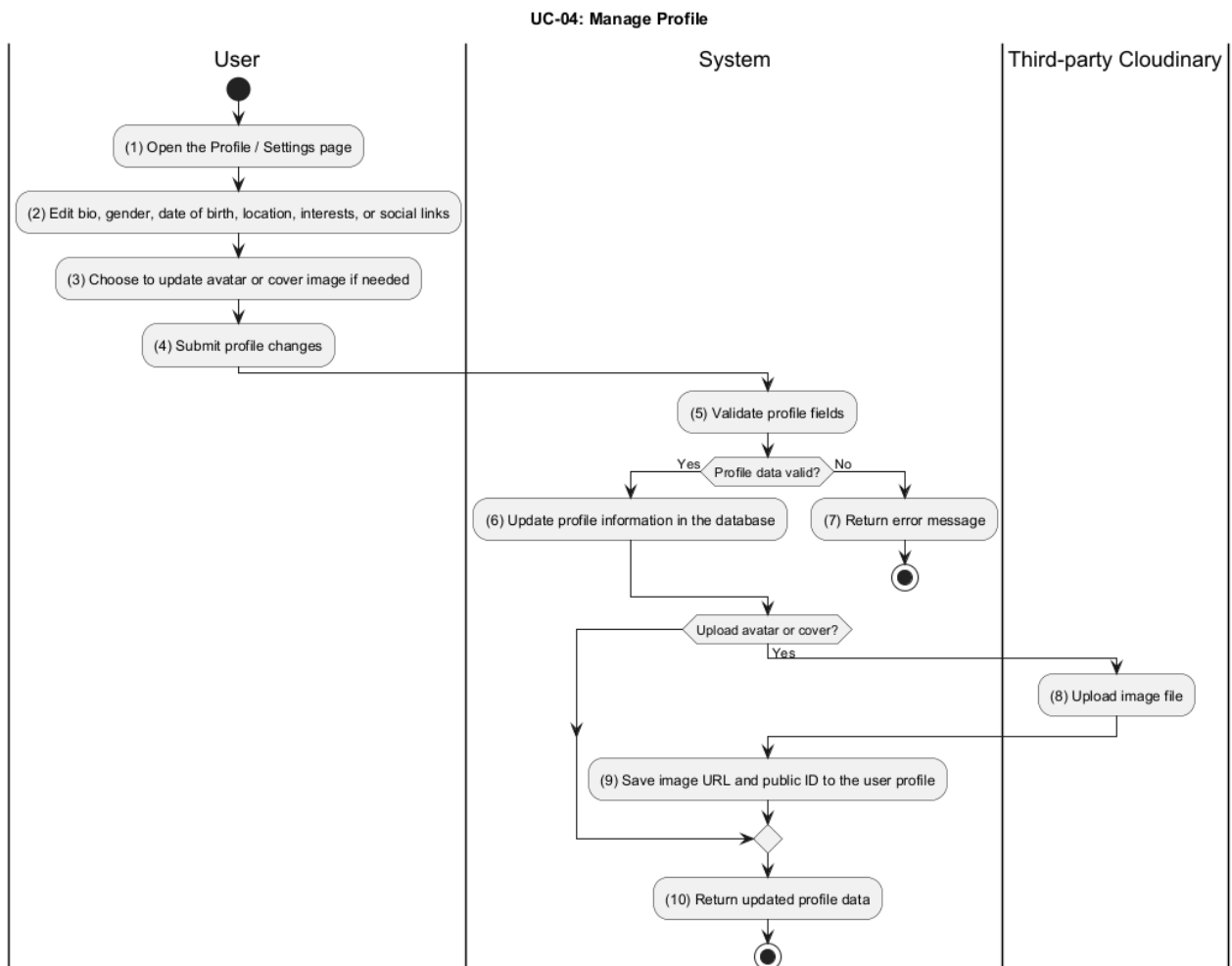


Figure 3.5: Activity Diagram for UC-04

ID	Activ-ity	Rule Type	Description
BR-UC04-01	(5)	Validation Rules	Validate Profile Fields: - The controller checks the JSON body for UserProfileUpdateRequest. - If binding fails, the controller returns a bad request response. bio is limited by DTO validation to 500 characters. social_links.website must be a valid URL when provided.
BR-UC04-02	(6)	Workflow Rules	Update Profile Data: - The service loads the current user and updates RoleContent.AsUser. - Empty string values clear the corresponding profile field. - The service accepts updates for gender, date_of_birth, location, interests, and social_links.
BR-UC04-03	(7)	Validation Rules	Reject Invalid Profile Values: - If gender is not valid, the service returns an invalid gender error. - If date_of_birth cannot be parsed as 2006-01-02, the service returns an invalid date format error. - If the calculated age is below 13, the service returns an age-too-young error. - If the calculated age is above 150, the service returns an invalid birth date error. - If location is not a valid province, the service returns an invalid province error. - If more than 10 interests are submitted or any interest is invalid, the service returns an error.

Table 3.9: Business Rules for UC-04 - Manage Profile

ID	Activ-ity	Rule Type	Description
BR-UC04-04	(8)	Workflow Rules	Upload Profile Image: - When the user uploads an avatar or cover image, the controller sends the file to <i>cloudinary.UploadImages()</i>. - The service stores the returned image URL and PublicID in the user profile. - If an older image exists, the service deletes the old asset asynchronously using <i>cloudinary.Delete()</i>.
BR-UC04-05	(9)	Workflow Rules	Save Image Metadata: - The service updates Avatar or Cover with the new image metadata. - The service persists the updated user profile in the repository.
BR-UC04-06	(10)	Message Rules	Return Updated Profile: - The system returns the refreshed user profile after saving changes. - If validation fails, the controller returns the corresponding error message.

Table 3.9: Business Rules for UC-04 - Manage Profile

3.1.5 UC-05 - Change Settings

Name	Change Settings
Description	This use case describes how a user changes account preferences such as privacy, notifications, and theme.
Actor	User
Trigger	When the user opens the settings page.
Pre-condition	The user is logged in to the system. The settings page is available to the user.
Post-condition	The selected preferences are saved. The account settings are updated for future sessions.

Table 3.10: Use Case Specification - Change Settings

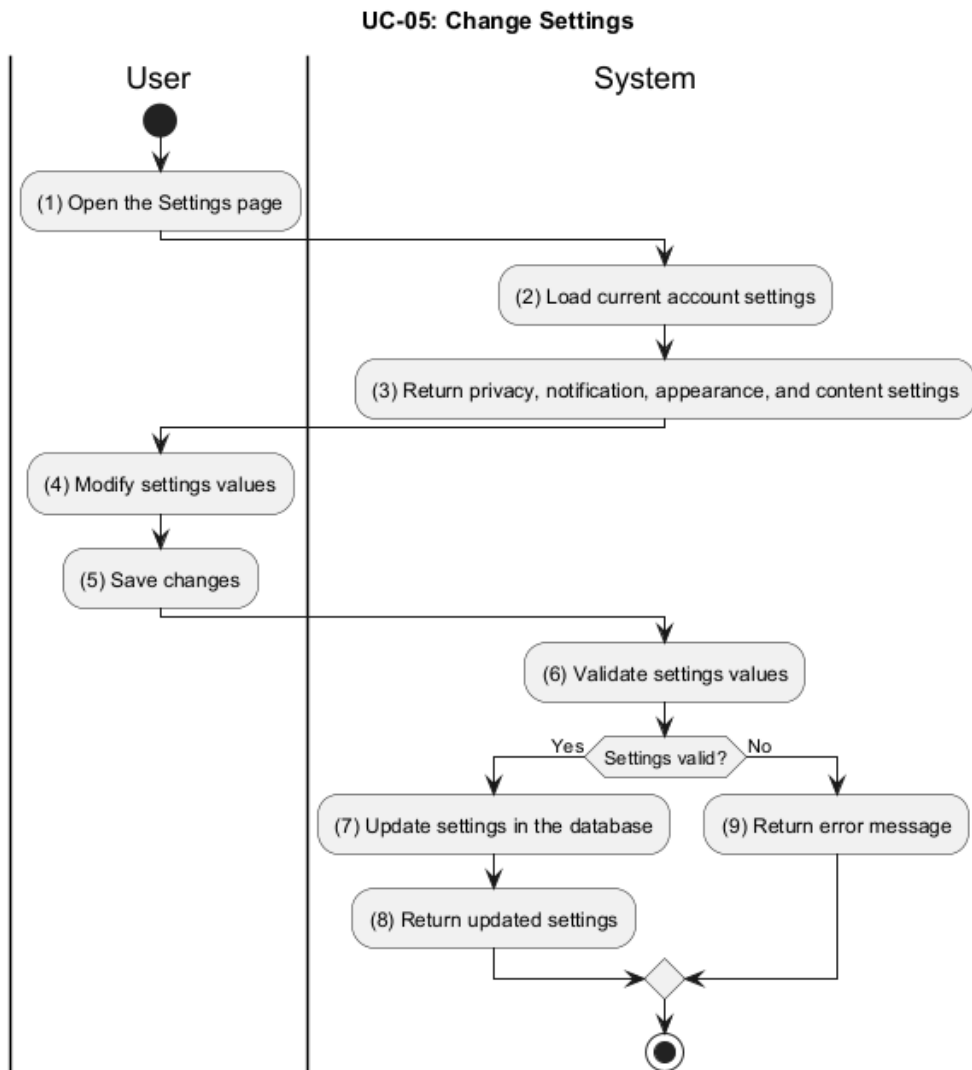


Figure 3.6: Activity Diagram for UC-05

ID	Activ-ity	Rule Type	Description
BR-UC05-01	(2)	Workflow Rules	Load Current Settings: - The system loads the current user's settings from the database using <i>GetSettings()</i>. - If the user has no settings record, the service returns default settings through <i>FromSettings()</i>.

Table 3.11: Business Rules for UC-05 - Change Settings

ID	Activ- ity	Rule Type	Description
BR-UC05-02	(6)	Validation Rules	Validate Settings Values: - The system checks the JSON body for <code>UpdateSettingsRequest</code>. - If binding fails, the controller returns a bad request response. theme must be one of <code>light</code>, <code>dark</code>, or <code>auto</code>. font_size must be one of <code>small</code>, <code>medium</code>, or <code>large</code>.
BR-UC05-03	(7)	Workflow Rules	Update Settings in Database: - The service loads the current user and initializes Settings when it is nil. - The service updates only the settings fields provided in the request. - The service persists the updated user record using the user repository.
BR-UC05-04	(8)	Message Rules	Return Updated Settings: - The controller returns the refreshed settings after the database update succeeds. - The response includes the updated appearance, notifications, privacy, and content values.
BR-UC05-05	(9)	Message Rules	Return Error Message: If validation fails or the service returns an error, the controller returns the corresponding error message.

Table 3.11: Business Rules for UC-05 - Change Settings

3.1.6 UC-06 - View Activity History

Name	View Activity History
Description	This use case describes how a user reviews past posts, comments, and interactions.
Actor	User
Trigger	When the user opens the activity history page.
Pre-condition	The user is logged in to the system. The user has activity records in the system.
Post-condition	The activity history is displayed to the user. The user can review prior activity.

Table 3.12: Use Case Specification - View Activity History

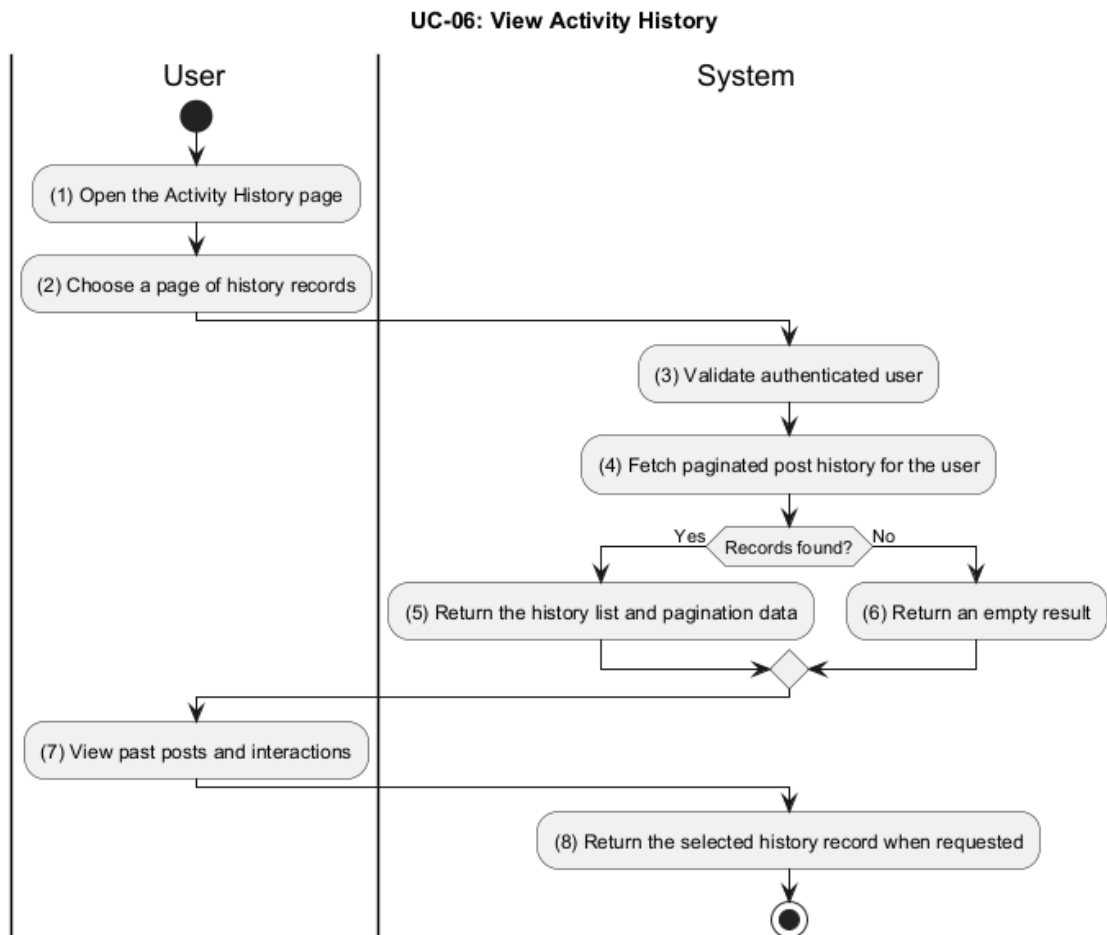


Figure 3.7: Activity Diagram for UC-06

ID	Activ- ity	Rule Type	Description
BR-UC06-01	(3)	Authorization Rules	Validate Authenticated User: - if authUser is missing, then the controller returns a forbidden response. - if authUser exists, then the system continues to fetch the activity history for that user.
BR-UC06-02	(4)	Workflow Rules	Fetch Paginated History: - if user__id does not match the authenticated user, then the service returns a forbidden error. - if the request is valid, then the service calls <i>GetPostHistoryByUserID()</i> to fetch the user's post history. - if page or page_size is invalid, then the controller uses defaults of 1 and 10.
BR-UC06-03	(5–6)	Message Rules	Return History List or Empty Result: - if records are found, then the controller returns the history list together with pagination data. - if no records are found, then the service returns an empty result with pagination data.
BR-UC06-04	(7)	Authorization Rules	Check Record Ownership: - if the requester opens a selected history record, then the service checks that requesterID matches the stored UserID. - if the requester does not own the record, then the service returns a forbidden error.
BR-UC06-05	(8)	Message Rules	Return Selected History Record: - if the requester owns the record, then the system returns the selected history item. - the returned item is converted with <i>FromPostHistory()</i> before being sent to the client.

Table 3.13: Business Rules for UC-06 - View Activity History

3.1.7 UC-07 - Follow / Unfollow User

Name	Follow / Unfollow User
Description	This use case describes how a user subscribes to or unsubscribes from another user's activity feed.
Actor	User
Trigger	When the user clicks the follow or unfollow action on another profile.
Pre-condition	The user is logged in to the system. The target profile is available for follow actions.
Post-condition	The follow relationship is created or removed. The user's feed and follower lists are updated.

Table 3.14: Use Case Specification - Follow / Unfollow User

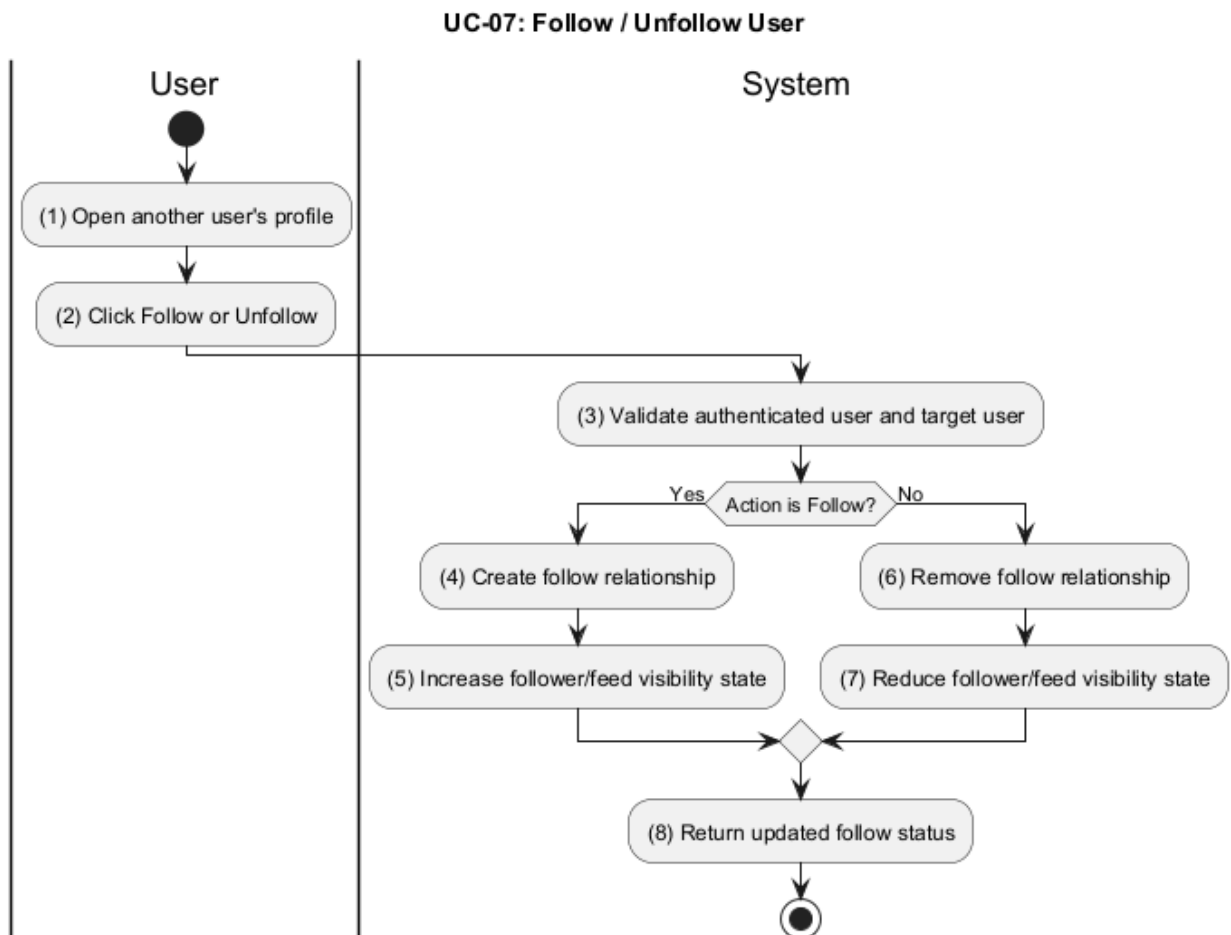


Figure 3.8: Activity Diagram for UC-07

Activ-ity	BR Code	Description
(3-6)	BR1	Backend Rule Status: • No follow or unfollow route, controller, or service is present in the backend code. • The repository currently has no server-side business rules to define for this use case. • This use case remains documented-only until a backend implementation is added.

Table 3.15: Business Rules for UC-07 - Follow / Unfollow User

3.1.8 UC-08 - Browse Post

Name	Browse Content
Description	This use case describes how a visitor or user explores communities and public posts.
Actor	Guest User / User
Trigger	When the user opens the forum homepage or a community page.
Pre-condition	The browsing page is accessible. The user may be logged in or not logged in.
Post-condition	Public posts and community content are shown. The user can continue browsing or open a post.

Table 3.16: Use Case Specification - Browse Post

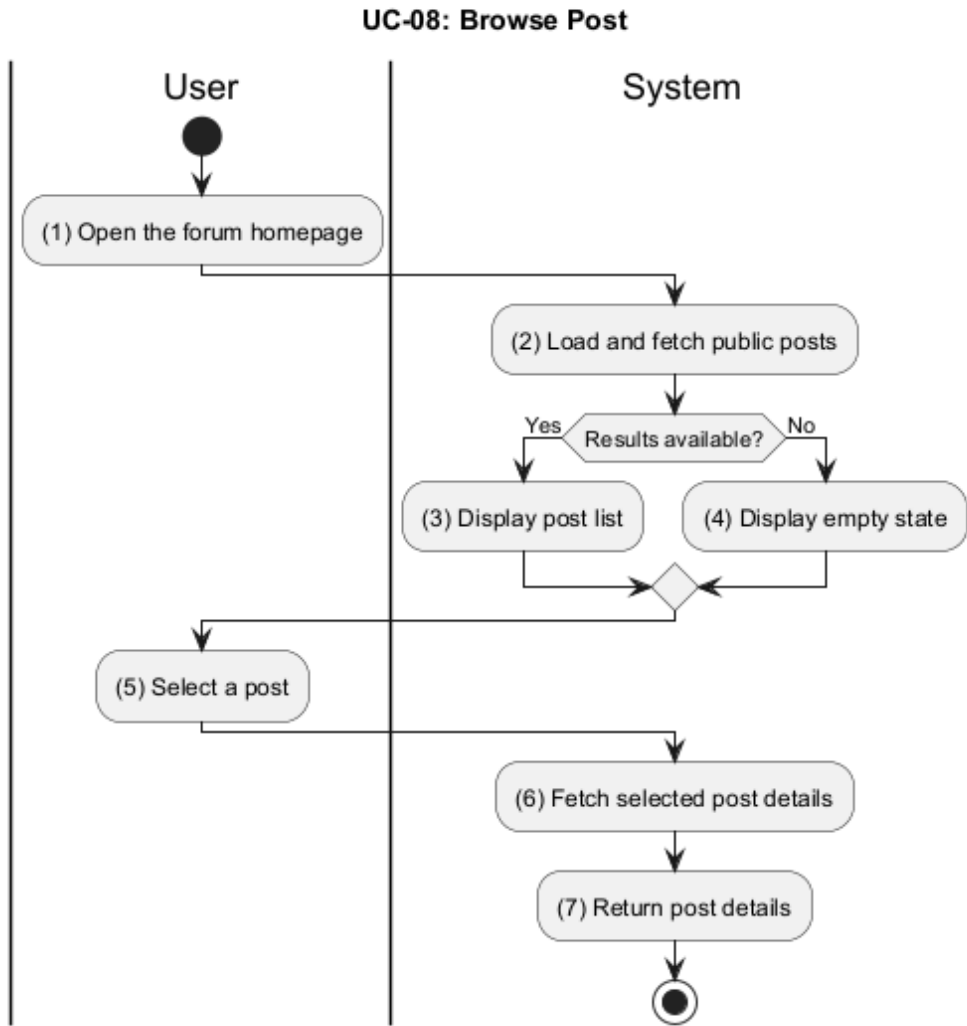


Figure 3.9: Activity Diagram for UC-08

ID	Activ-ity	Rule Type	Description
BR-UC08-01	(2)	Workflow Rules	Fetch Posts from Database: - The system calls <i>GetPosts()</i> to fetch public posts from the database. - The system fetches authors and communities to enrich the post response. - if totalPosts == 0, then return an empty state instead of an error.

Table 3.17: Business Rules for UC-08 - Browse Post

ID	Activ- ity	Rule Type	Description
BR-UC08-02	(3-4)	Message Rules	Return Content List or Empty State: - if records exist, then return the post list with pagination data. - if no records exist, then return an empty result with pagination data.
BR-UC08-03	(6)	Workflow Rules	View Selected Post: - if the user selects a post, then the system loads the post details using <i>GetPostById()</i>. - if userID != null, then the system also loads userVote and userPollVoteIDs.
BR-UC08-04	(6)	Validation Rules	Validate Selected Post Visibility: - if the selected post is hidden and post.AuthorID != userID, then return post-not-found. - if the selected post is hidden and post.AuthorID == userID, then allow the request to continue.
BR-UC08-06	(7)	Message Rules	Return Selected Post: - The system returns the selected post to the user. - if no post is selected, then the browse list remains visible and no error is returned.

Table 3.17: Business Rules for UC-08 - Browse Post

3.1.9 UC-09 - Search Content

Name	Search Content
Description	This use case describes how a user searches for posts, users, or communities by keyword.
Actor	User
Trigger	When the user enters a search term and submits the search.
Pre-condition	The search page is available. The system has searchable content.
Post-condition	Matching results are returned to the user. The user can open a result or refine the search.

Table 3.18: Use Case Specification - Search Content

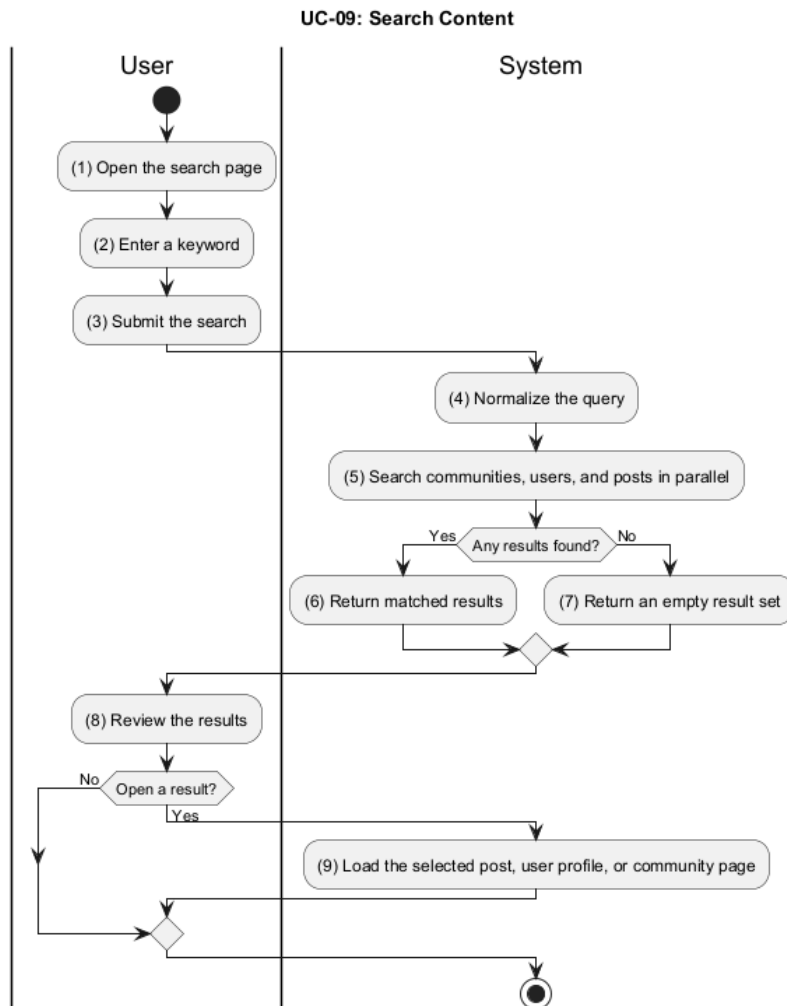


Figure 3.10: Activity Diagram for UC-09

ID	Activ- ity	Rule Type	Description
BR-UC09-01	(4)	Validation Rules	Normalize the Query: - The system decodes the query and removes optional prefixes such as <code>c/</code>, <code>lk/</code>, and <code>u/</code>. - if <code>cleanQuery == null</code> or <code>cleanQuery.trim() == ""</code>, then the system keeps the result sets empty.
BR-UC09-02	(5)	Workflow Rules	Search Communities, Users, and Posts in Parallel: - The system calls <i>GetCommunitiesFilter()</i>, <i>GetUsers()</i>, and <i>GetPosts()</i> through the frontend service layer. - The three requests run in parallel with the same cleaned query. - if any request fails, then the search page clears the result lists and leaves the page in the empty-state path.
BR-UC09-03	(6–7)	Message Rules	Return Matched Results or Empty State: - if any result list contains items, then the system renders the matching posts, communities, and users. - if all result lists are empty, then the system shows the empty-state message instead of an error.
BR-UC09-04	(9)	Redirect Rules	Open Selected Result: - if the user selects a post, then the system opens the generated post URL. - if the user selects a community, then the system navigates to the community page using <code>/lk/{communityName}</code>. - if the user selects a user, then the system navigates to the profile page using <code>/profile/{username}</code>.

Table 3.19: Business Rules for UC-09 - Search Content

3.1.10 UC-10 - Create Post

Name	Create Post
Description	This use case describes how a member creates and submits a new post in a community.
Actor	User
Trigger	When the user clicks the “Create Post” action.
Pre-condition	The user is logged in to the system. The user is a member of the selected community.
Post-condition	The post is stored successfully. The post appears in the community or enters moderation if required.

Table 3.20: Use Case Specification - Create Post

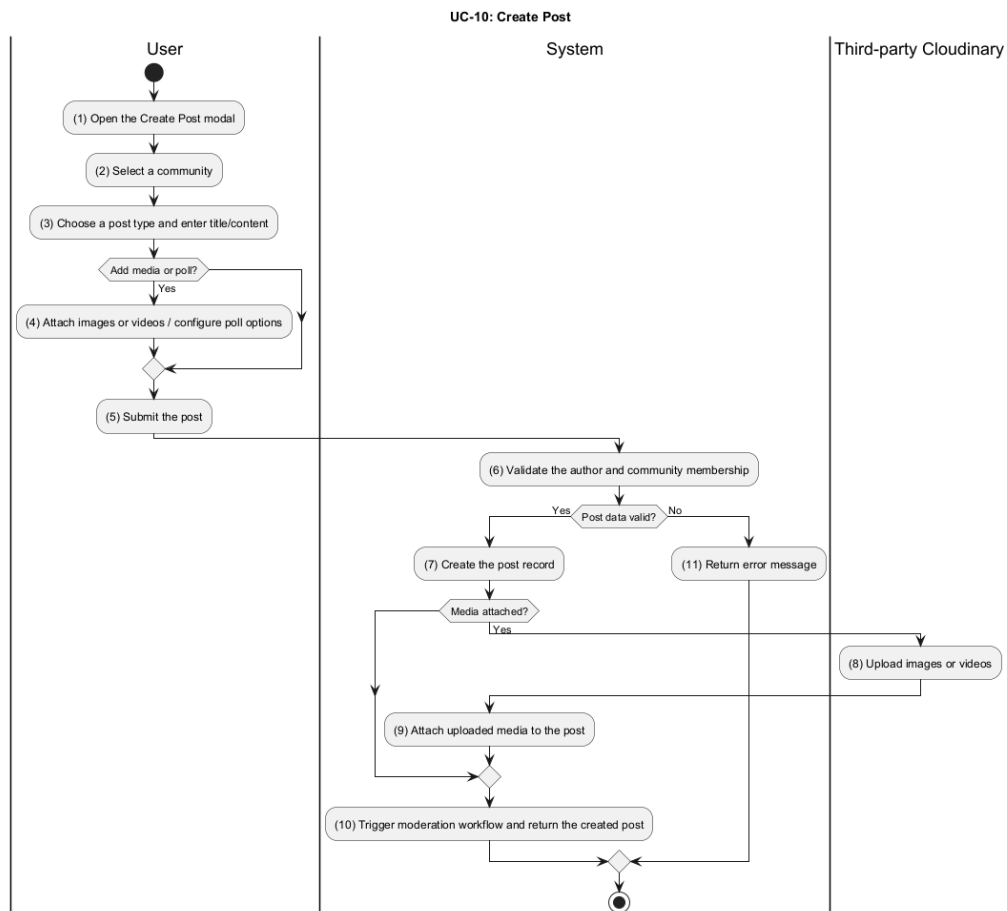


Figure 3.11: Activity Diagram for UC-10

ID	Activ- ity	Rule Type	Description
BR-UC10-01	(6)	Validation Rules	Validate Create Post Request: • if authUser == null, then the controller returns a forbidden response. • if JSON binding fails for CreatePostRequest, then the controller returns a bad request response. • the DTO requires community_id, title, and type; if any required field is missing, binding fails. • type must be one of text, poll, image, or video.
BR-UC10-02	(6)	Authorization Rules	Validate Author and Community Membership: • if communityID == null or cannot be parsed to an ObjectID, then the service returns an invalid ID error. • if the community does not exist, then the service returns the repository error. • if the author does not exist, then the service returns the repository error. • if communityRepo.IsUserBanned() == true, then the service returns a banned-from-community error.

Table 3.21: Business Rules for UC-10 - Create Post

ID	Activ- ity	Rule Type	Description
BR-UC10-03	(6)	Workflow Rules	Determine Moderation State: • if author.Role == AdminRole, then set initialStatus = ModerationSkipped. • if community.CreateByID == userID, then set initialStatus = ModerationSkipped. • if any community moderator has mod.UserID == userID, then set initialStatus = ModerationSkipped. • if community.Setting.PostRequireApproval == true and the author is not admin/mod/creator, then set initialStatus = ModerationPending. • otherwise, set initialStatus = ModerationPending for AI moderation.
BR-UC10-04	(7)	Workflow Rules	Create Post Record: • The system creates a Post with AuthorID, CommunityID, Title, Type, Content.text, Tags, and ModerationStatus. • The system initializes VotesCount, CommentsCount, HotScore, and flag fields such as IsDeleted, IsHidden, IsDraft, and IsBan. • if type == poll and req.Poll != null, then the system creates poll options and attaches poll data to the post content. • The system persists the post through the post repository.

Table 3.21: Business Rules for UC-10 - Create Post (Continued)

ID	Activ- ity	Rule Type	Description
BR-UC10-05	(8–9)	Workflow Rules	Attach Media to Post: - if images are attached, then the system uploads them using <i>UploadImages()</i> and pushes them into content.images. - if videos are attached, then the system uploads them using <i>UploadVideos()</i> and pushes them into content.videos. - if media upload fails, then the system returns the upload error and does not complete the attach step.
BR-UC10-06	(10)	Workflow Rules	Trigger Moderation and Return Post: - The system increments the author's post count after the post is created. - The system auto-upvotes the post for the creator using <i>VoteOnTargetByAuthor()</i>. - The system publishes a post-created event to trigger moderation workflow. - The system returns the created post using <i>GetPostByID()</i>.
BR-UC10-07	(11)	Message Rules	Return Error Message: if validation, authorization, or persistence fails, then the controller returns the corresponding error message.

Table 3.21: Business Rules for UC-10 - Create Post

3.1.11 UC-11 - Edit Post

Name	Edit Post
Description	This use case describes how a member modifies an existing post and preserves edit history.
Actor	User
Trigger	When the user opens an existing post and chooses “Edit”.
Pre-condition	The user is logged in to the system. The user owns the post or has edit permission.
Post-condition	The edited post is saved successfully. The edit history is retained.

Table 3.22: Use Case Specification - Edit Post

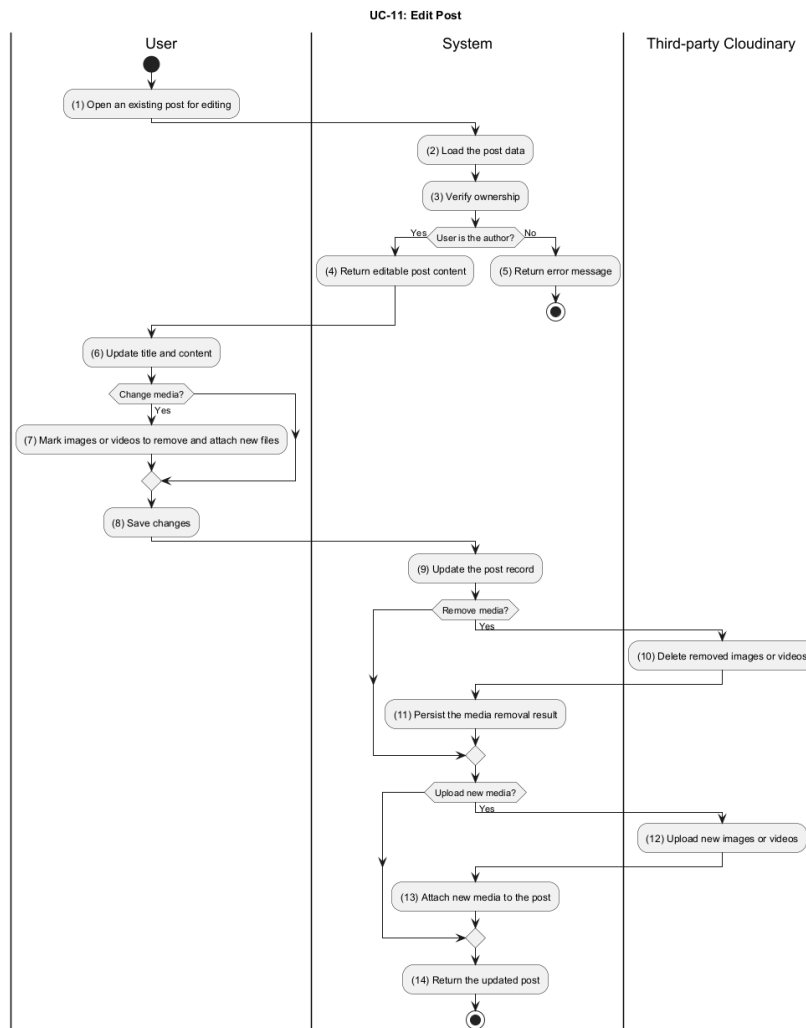


Figure 3.12: Activity Diagram for UC-11

ID	Activ- ity	Rule Type	Description
BR-UC11-01	(4)	Authorization Rules	Load Post and Verify Ownership: - The controller requires an authenticated user for the edit request. - if authUser == null, then the controller returns a forbidden response. - if JSON binding fails for UpdatePostRequest, then the controller returns a bad-request response. - The service loads the post by ID and if post.AuthorID != userID, then the service returns a forbidden error.
BR-UC11-02	(9)	Workflow Rules	Handle Media Changes Through Post Media Endpoints: - if the user removes images, then the system calls <i>RemoveImagesFromPost()</i> after validating public_ids and author ownership. - if the user adds images, then the system calls <i>AddImagesToPost()</i> and uploads the files with <i>UploadImages()</i> before pushing them into content.images. - if the user removes videos, then the system calls <i>RemoveVideosFromPost()</i> and deletes the matched Cloudinary assets after the database update. - if the user adds videos, then the system calls <i>AddVideosToPost()</i> and uploads the files with <i>UploadVideos()</i> before pushing them into content.videos. - if new videos are added, then the system also sets type = video.

Table 3.23: Business Rules for UC-11 - Edit Post

ID	Activ- ity	Rule Type	Description
BR-UC11-04	(10–13)	Workflow Rules	Persist Edit Result and Media Side Effects: - The system updates the post record with the new content and updated_at timestamp. - The system publishes a PostUpdatedEvent for moderation. - if image or video upload fails, then the system returns the error and does not finish the edit flow. - if video removal is requested, then the system only removes IDs that already exist in the post.
BR-UC11-05	(14)	Message Rules	Return the Updated Post: - The system returns the refreshed post after the update or media change succeeds. - if any validation, ownership, or persistence step fails, then the controller returns the corresponding error message.

Table 3.23: Business Rules for UC-11 - Edit Post

3.1.12 UC-12 - Delete Post

Name	Delete Post
Description	This use case describes how a member or moderator removes a post from public view using soft delete.
Actor	User / Moderator
Trigger	When the user clicks the “Delete” action on a post.
Pre-condition	The user is logged in to the system. The user has permission to delete the post.
Post-condition	The post is marked as deleted. The post is hidden from public view.

Table 3.24: Use Case Specification - Delete Post

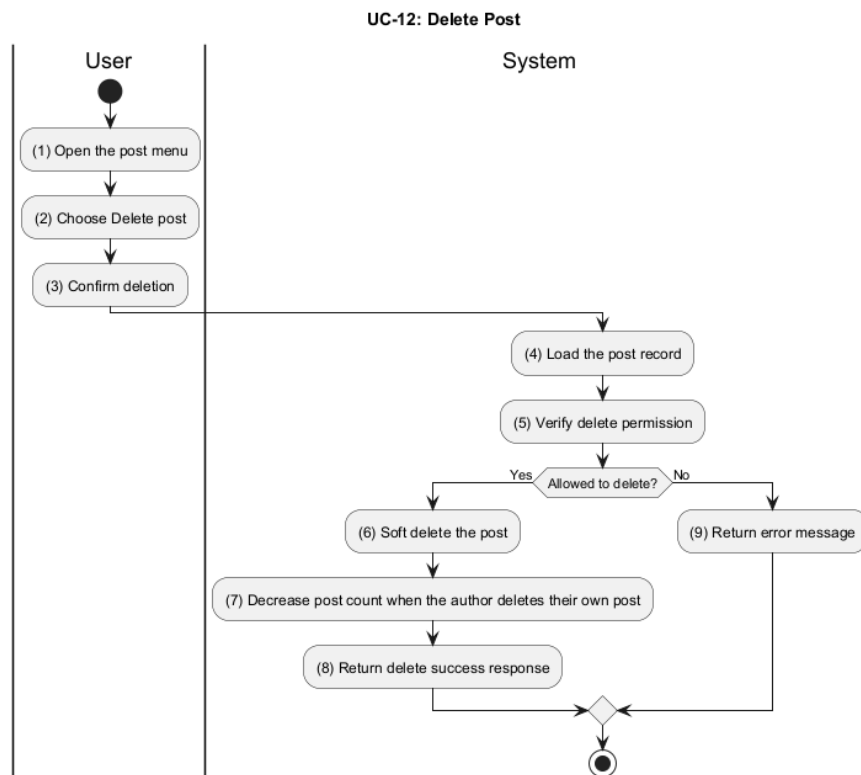


Figure 3.13: Activity Diagram for UC-12

ID	Activ- ity	Rule Type	Description
BR-UC12-01	(4)	Authorization Rules	Verify Delete Permission: - The system requires an authenticated user for the delete request. - if authUser == null, then the controller returns an unauthorized response. - The service loads the post by ID and if the post no longer exists, then it returns a post-not-found error. - if post.AuthorID == userID, then the requester may delete their own post. - if post.AuthorID != userID, then the service checks whether the user is an admin, the community creator, or a community moderator. - if none of those permissions exist, then the service returns a forbidden error.
BR-UC12-02	(6-7)	Workflow Rules	Soft Delete the Post Record and Adjust Counters: - The system marks the post as deleted in storage through <i>SoftDelete()</i>. - if the author deletes their own post, then the system decreases the author's post count by one. - if the requester is not the author, then the system does not change the author's post count.
BR-UC12-03	(8)	Message Rules	Return Success or Error Response: if any authorization or persistence step fails, then the controller returns the corresponding error message.
BR-UC12-04	(9)	Message Rules	Return Success or Error Response: if any authorization or persistence step fails, then the system returns the corresponding error message.

Table 3.25: Business Rules for UC-12 - Soft Delete Post

3.1.13 UC-13 - Post Comment

Name	Post Comment
Description	This use case describes how a user adds a direct comment to a post.
Actor	User
Trigger	When the user submits a comment on a post.
Pre-condition	The user is logged in to the system. The target post is visible and commentable.
Post-condition	The comment is created and attached to the post. The comment count is updated.

Table 3.26: Use Case Specification - Post Comment

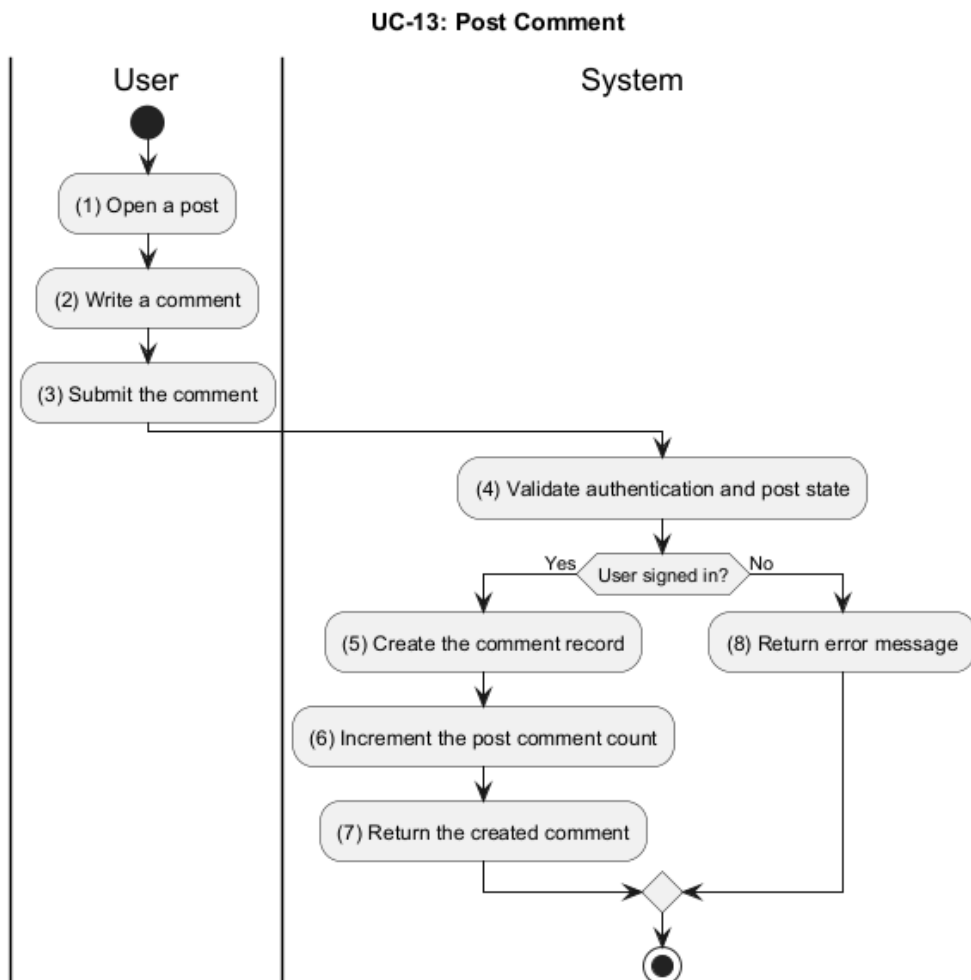


Figure 3.14: Activity Diagram for UC-13

ID	Activ- ity	Rule Type	Description
BR-UC13-01	(4)	Validation Rules	Validate Request, Post State, and Access: - The controller binds the request to <code>CreateCommentRequest</code>. - if JSON binding fails, then the controller returns a bad-request response. post_id and content are required by the request DTO. - The service loads the target post by post_id; if the post cannot be found, then the comment flow stops. - The system checks whether the user is muted or banned in the post's community. - if authUser == null, then the controller returns a forbidden response. - if the user is muted or banned, then the service returns the corresponding access error.
BR-UC13-02	(4)	Message Rules	Return Bad-Request Message: if request binding fails, then the system returns the bad-request message for the comment form.
BR-UC13-03	(5–6)	Workflow Rules	Create Comment and Update Counters: - The system creates the comment with the current user as author, the selected post as target, and the optional parent comment. - The system stores the new comment with IsDeleted = false. - The system increments the post comment count and the user's comment count. - The system publishes the comment-created event for downstream processing.

Table 3.27: Business Rules for UC-13 - Post Comment

ID	Activ-ity	Rule Type	Description
BR-UC13-04	(7)	Message Rules	Return Created Comment Message: The system returns Comment created successfully with the created comment payload.

Table 3.27: Business Rules for UC-13 - Post Comment

3.1.14 UC-14 - Reply to Comment

Name	Reply to Comment
Description	This use case describes how a user replies to an existing comment in a nested discussion thread.
Actor	User
Trigger	When the user submits a reply to a comment.
Pre-condition	The user is logged in to the system. The parent comment exists and can receive replies.
Post-condition	The reply is created as a nested comment. The discussion thread is updated.

Table 3.28: Use Case Specification - Reply to Comment

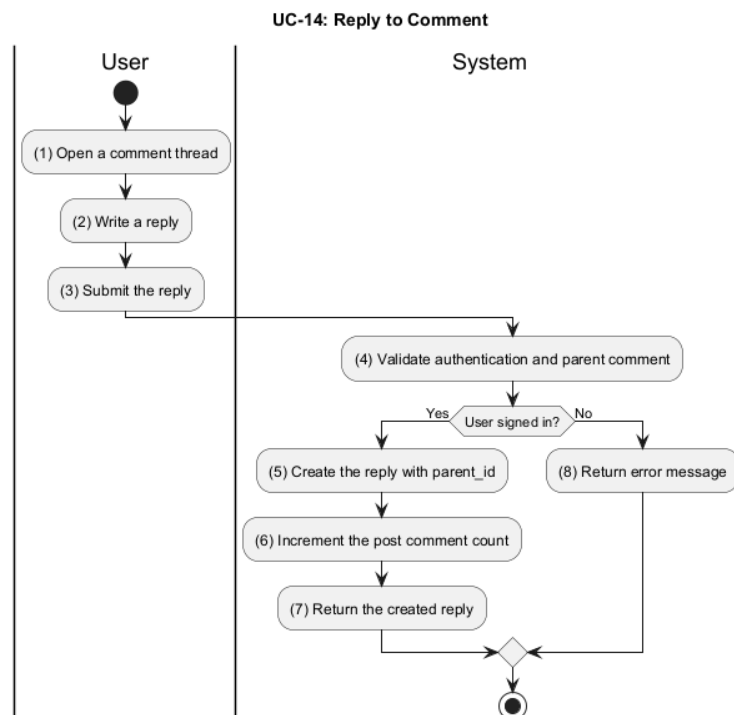


Figure 3.15: Activity Diagram for UC-14

ID	Activ- ity	Rule Type	Description
BR-UC14-01	(4)	Validation Rules	Validate Parent Reply Request and Access: • The controller binds the request to <code>CreateCommentRequest</code>. • if JSON binding fails, then the controller returns a bad-request response. • post_id and content are required by the request DTO. • The controller requires an authenticated user before creating the reply. • if authUser == null, then the controller returns a forbidden response. • The service loads the target post by post_id; if the post cannot be found, then the reply flow stops. • if parent_id is provided, then the service parses it and uses it as the reply parent when valid. • The system checks whether the user is muted or banned in the post's community. • if the user is muted or banned, then the service returns the corresponding access error.
BR-UC14-02	(4)	Message Rules	Return Bad-Request Message: • if request binding fails, then the system returns the bad-request message for the reply form.

Table 3.29: Business Rules for UC-14 - Reply to Comment

BR-UC14-03	(5-6)	Workflow Rules	Create Reply and Update Counters: • The system creates the reply with the current user as author, the selected post as target, and the optional parent comment. • The system stores the new reply with IsDeleted = false. • The system increments the post comment count and the user's comment count. • The system publishes the comment-created event for downstream processing.
BR-UC14-04	(7)	Message Rules	Return Created Reply Message: • The system returns Comment created successfully with the created reply payload.

Table 3.29: Business Rules for UC-14 - Reply to Comment

3.1.15 UC-15 - Vote on Content

Name	Vote on Content
Description	This use case describes how a user upvotes or downvotes a post or comment.
Actor	User
Trigger	When the user selects an upvote or downvote action.
Pre-condition	The user is logged in to the system. The target content is available for voting.
Post-condition	The vote is recorded or updated. The content score is recalculated.

Table 3.30: Use Case Specification - Vote on Content

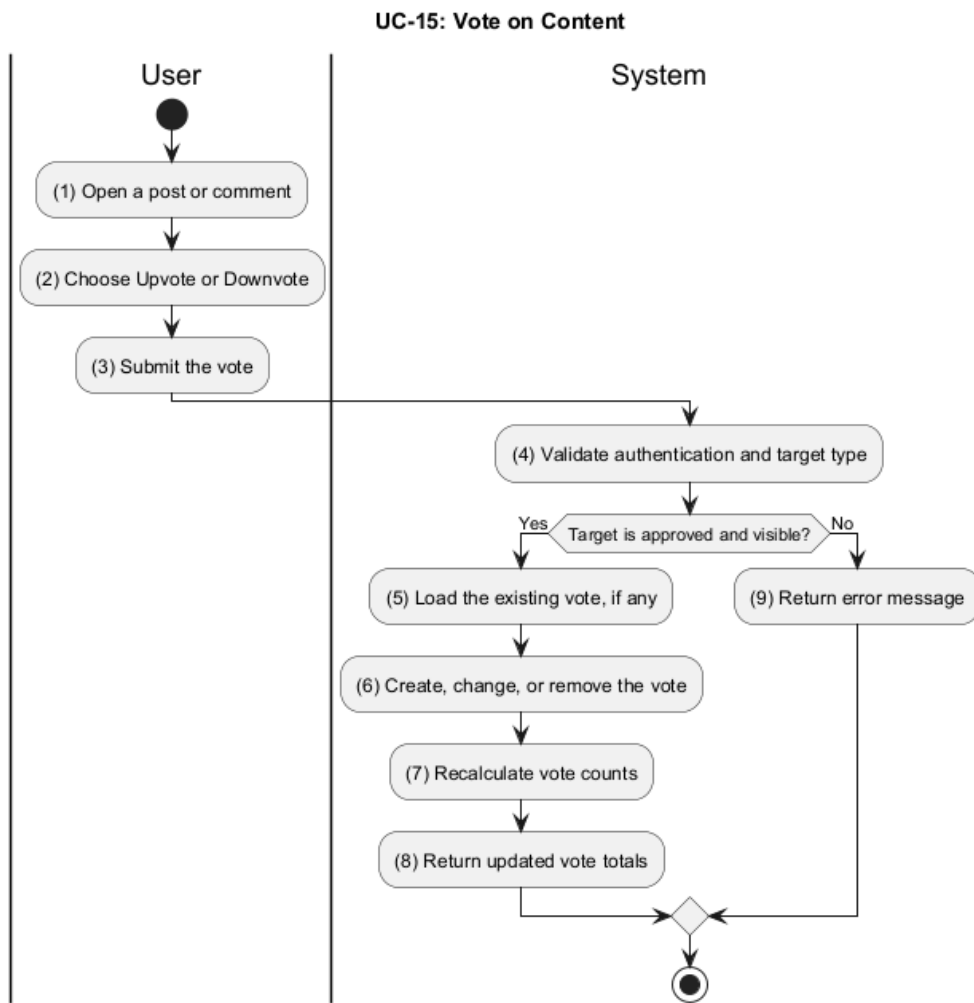


Figure 3.16: Activity Diagram for UC-15

ID	Activ- ity	Rule Type	Description
BR-UC15-01	(4)	Validation Rules	Validate Authentication, Target Type, and Vote Payload: - The controller requires an authenticated user before casting or removing a vote. - if authUser == null, then the controller returns an unauthorized response. - The controller accepts only post or comment as target_type. - if the target type is invalid, then the controller returns a bad-request response. - The controller binds the request body to PostVoteRequest. - if JSON binding fails or value == null, then the controller returns a bad-request response.
BR-UC15-02	(4)	Workflow Rules	Check Target Visibility and Load Existing Vote: - For post voting, the system loads the post and allows voting only when the moderation status is approved or skipped. - For comment voting, the system loads the comment and does not apply a moderation gate. - The system loads the existing vote, if any, before applying the new vote action.
BR-UC15-03	(4)	Message Rules	Return Bad-Request Message: if the target type is invalid or the vote payload is malformed, then the system returns the bad-request message.

Table 3.31: Business Rules for UC-15 - Vote on Content

ID	Activ- ity	Rule Type	Description
BR-UC15-04	(5-7)	Workflow Rules	Create, Change, or Remove the Vote: - if no previous vote exists, then the system creates a new vote record. - if the new value matches the previous value, then the system removes the vote. - if the new value differs from the previous value, then the system updates the existing vote value. - The vote repository updates the target's vote counters inside a transaction. - For post votes, the repository also recalculates hot_score from the new totals.
BR-UC15-05	(8)	Message Rules	Return Updated Vote Totals: The system returns the updated upvote, downvote, and score totals after the transaction completes.
BR-UC15-06	(9)	Message Rules	Return Vote Error Message: if authentication, moderation, or persistence fails, then the controller returns the corresponding error message.

Table 3.31: Business Rules for UC-15 - Vote on Content

3.1.16 UC-16 - Participate in Poll

Name	Participate in Poll
Description	This use case describes how a user casts a vote in a community poll.
Actor	User
Trigger	When the user submits a poll vote.
Pre-condition	The user is logged in to the system. The poll is open for voting.
Post-condition	The selected poll option is recorded. The poll results are updated.

Table 3.32: Use Case Specification - Participate in Poll

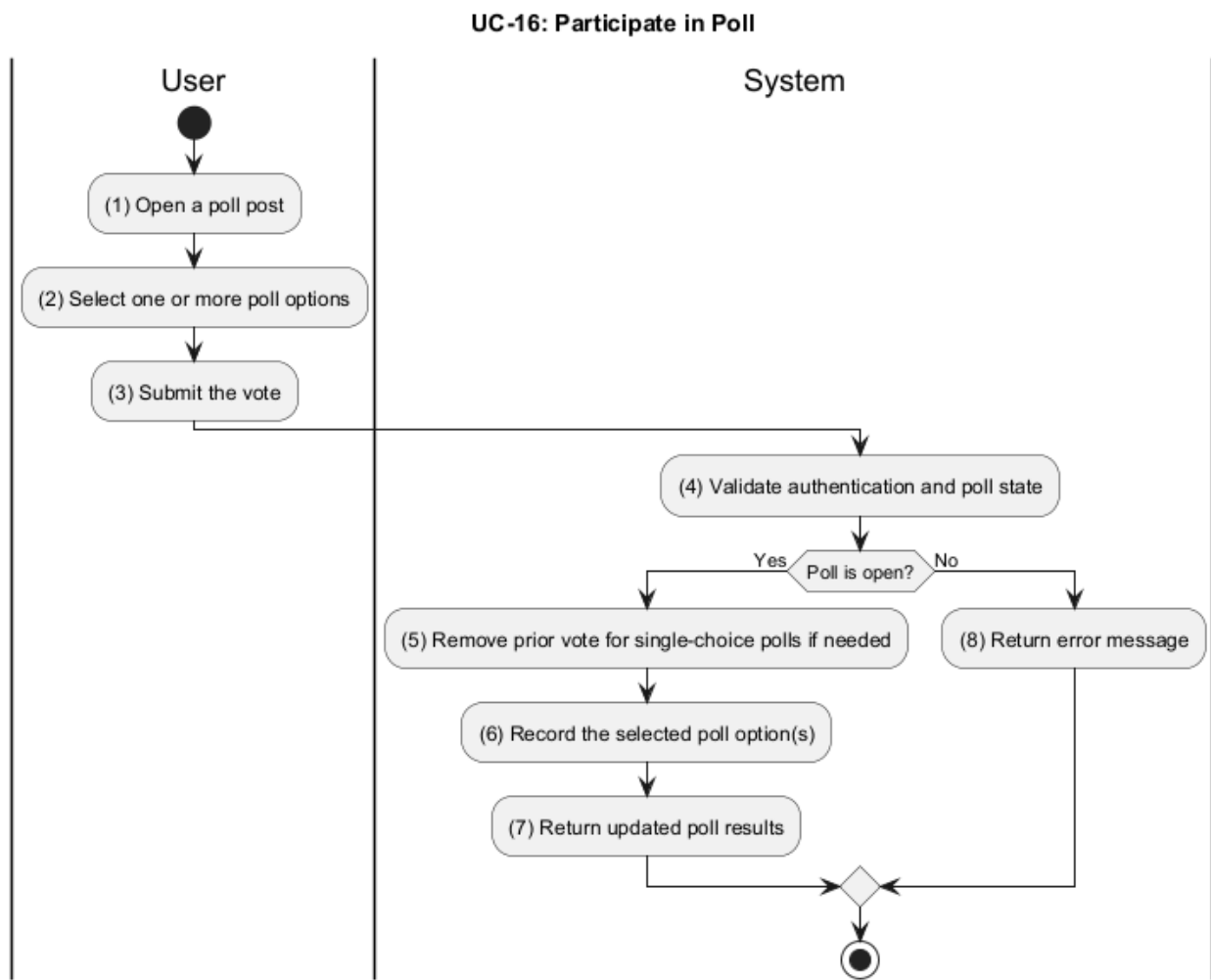


Figure 3.17: Activity Diagram for UC-16

ID	Activ- ity	Rule Type	Description
BR-UC16-01	(4)	Validation Rules	Validate Authentication, Poll State, and Vote Payload: - The controller requires an authenticated user before recording a poll vote. - if authUser == null, then the controller returns an unauthorized response. - The controller binds the request body to PollVoteRequest. - if JSON binding fails or option_id is missing, then the controller returns a bad-request response. - The service loads the target post and requires it to contain poll content. - if the post cannot be found, is not a poll, or does not contain poll data, then the service returns an error.
BR-UC16-02	(4)	Message Rules	Return Bad-Request Message: if the request payload is invalid, then the system returns the bad-request message.
BR-UC16-03	(5–6)	Workflow Rules	Record the Poll Vote: - The repository loads the user's existing poll votes for the post. - if the poll is single-choice and the user already voted another option, then the repository removes the previous vote before adding the new one. - if the selected option was already voted by the user, then the repository removes that specific vote instead of adding a duplicate. - The repository stores the new poll vote and increments the selected option vote count plus the poll total votes.

Table 3.33: Business Rules for UC-16 - Participate in Poll

ID	Activ-ity	Rule Type	Description
BR-UC16-04	(7)	Workflow Rules	Return Updated Poll Results: - The service reloads the poll and user vote IDs after the transaction completes. - The system returns the refreshed poll response with updated totals and user vote IDs.
BR-UC16-05	(7)	Message Rules	Return Created Poll Vote Message: The system returns Poll vote cast successfully with the updated poll payload.
BR-UC16-06	(8)	Message Rules	Return Poll Vote Error Message: if authentication, poll validation, or persistence fails, then the controller returns the corresponding error message.

Table 3.33: Business Rules for UC-16 - Participate in Poll

3.1.17 UC-17 - Save Post

Name	Save Post
Description	This use case describes how a user bookmarks a post for later viewing.
Actor	User
Trigger	When the user clicks the “Save” action on a post.
Pre-condition	The user is logged in to the system. The target post exists and is visible to the user.
Post-condition	The post is added to the user’s saved list. The saved state is available in the account.

Table 3.34: Use Case Specification - Save Post

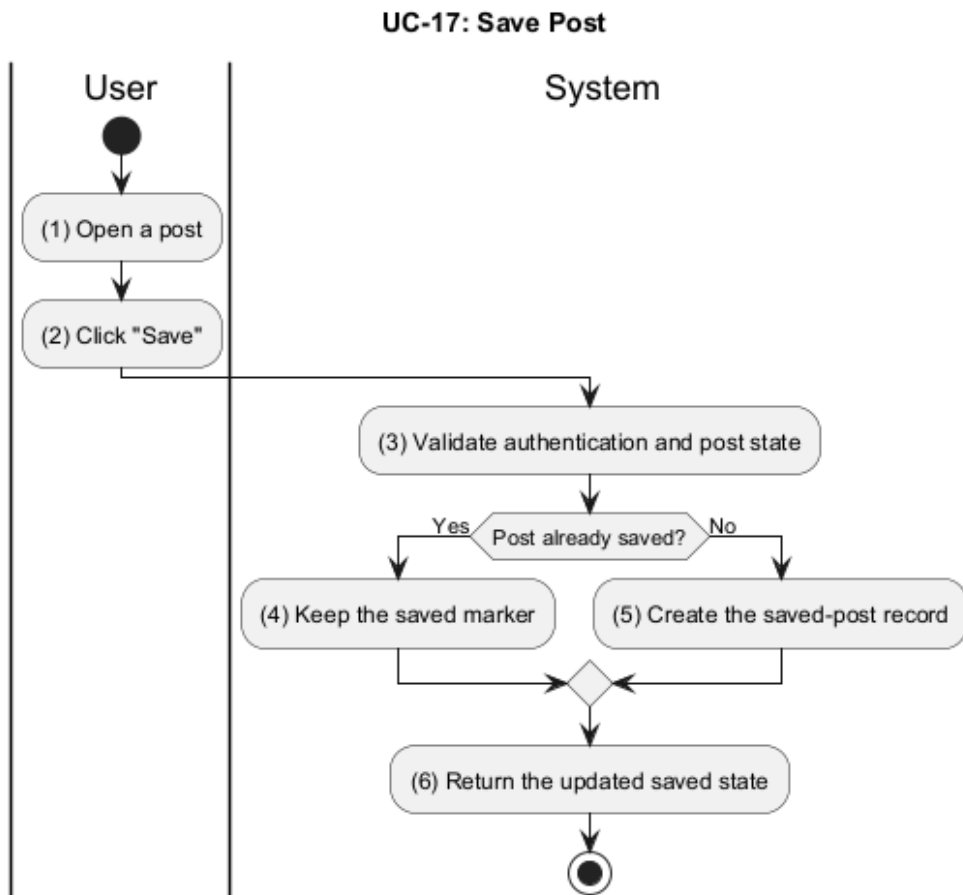


Figure 3.18: Activity Diagram for UC-17

ID	Activ-ity	Rule Type	Description
BR-UC17-01	(3)	Validation Rules	Validate Authentication and Post ID: - The controller requires an authenticated user before saving a post. - if <code>authUser == null</code>, then the controller returns a forbidden response. - The service parses the user ID and post ID to ObjectIDs. - if either ID cannot be parsed, then the service returns an invalid-ID error.

Table 3.35: Business Rules for UC-17 - Save Post

ID	Activ-ity	Rule Type	Description
BR-UC17-02	(3)	Workflow Rules	Check Post State and Save Relation: - The service loads the post before saving it. - if the post cannot be found, then the service returns the post lookup error. - The saved-post repository stores the relation with upsert semantics so an existing saved marker is kept instead of duplicated.
BR-UC17-03	(4–5)	Workflow Rules	Create or Keep the Saved Marker: - if the post is already saved by the user, then the system keeps the saved marker unchanged. - if the post is not yet saved, then the system creates the saved-post record.
BR-UC17-04	(6)	Message Rules	Return Updated Saved State: The system returns Post saved successfully after the save relation is stored.
BR-UC17-05	(6)	Message Rules	Return Save Error Message: if authentication, ID parsing, or post lookup fails, then the controller returns the corresponding error message.

Table 3.35: Business Rules for UC-17 - Save Post

3.1.18 UC-18 - Hide Post

Name	Hide Post
Description	This use case describes how a user hides a post from their personal feed.
Actor	User
Trigger	When the user clicks the “Hide” action on a post.
Pre-condition	The user is logged in to the system. The target post is visible in the user’s feed.
Post-condition	The post is hidden from the user’s feed. The hidden-post record is stored for the account.

Table 3.36: Use Case Specification - Hide Post

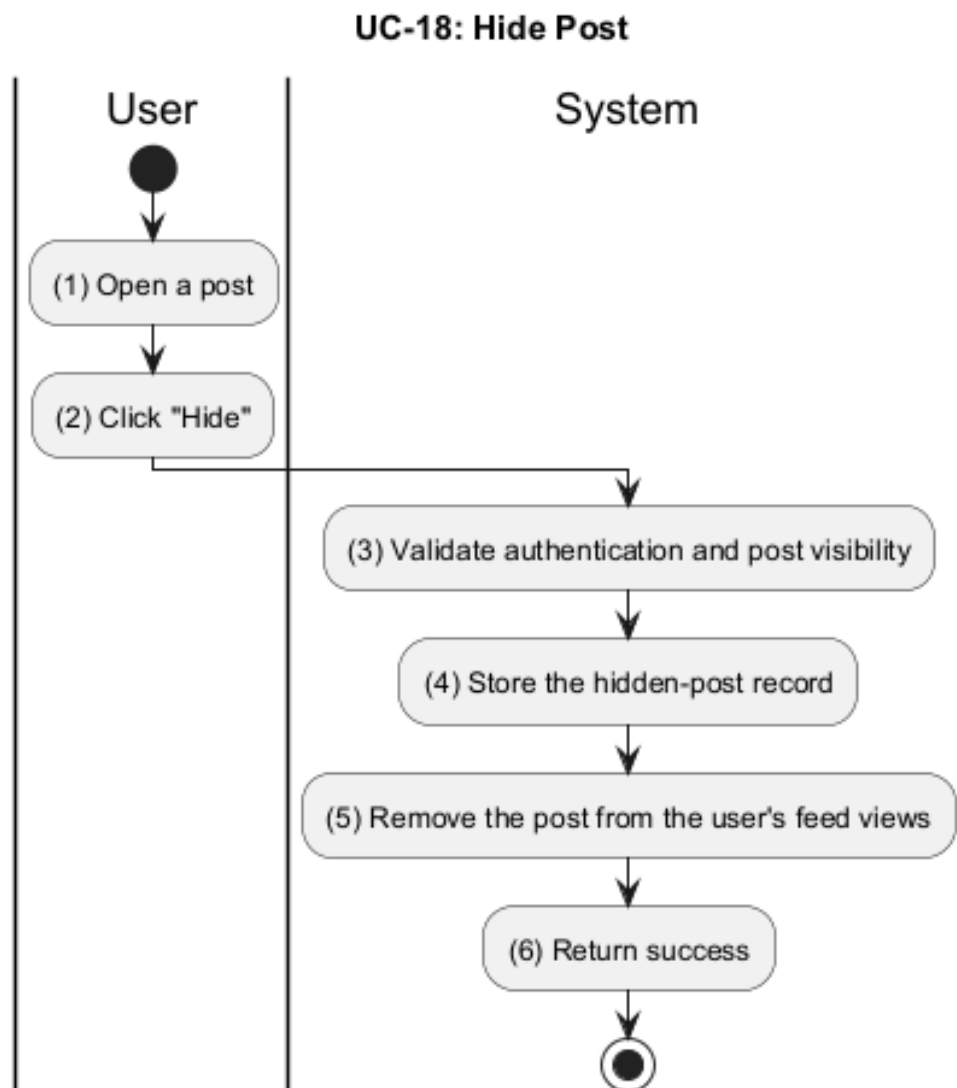


Figure 3.19: Activity Diagram for UC-18

ID	Activ- ity	Rule Type	Description
BR-UC18-01	(3)	Validation Rules	Validate Authentication and Post Ownership: - The controller requires an authenticated user before hiding a post. - if <code>authUser == null</code>, then the controller returns a forbidden response. - The service loads the post by ID before hiding it. - if the post cannot be found, then the service returns the post lookup error. - if the authenticated user is not the post author, then the service returns a forbidden error.
BR-UC18-02	(4-5)	Workflow Rules	Mark the Post Hidden and Remove It From Visible Feeds: - The service updates the post record by setting <code>is_hidden = true</code>. - The hidden marker keeps the post out of normal feed queries because feed filters exclude hidden posts.
BR-UC18-03	(6)	Message Rules	Return Hide Success Message: The system returns <code>Post hidden successfully</code> after the hidden state is stored.
BR-UC18-04	(6)	Message Rules	Return Hide Error Message: if authentication, ownership, or persistence fails, then the controller returns the corresponding error message.

Table 3.37: Business Rules for UC-18 - Hide Post

3.1.19 UC-19 - Manage Drafts

Name	Manage Drafts
Description	This use case describes how a user saves unfinished posts as drafts and publishes them later.
Actor	User
Trigger	When the user saves, edits, opens, or publishes a draft.
Pre-condition	The user is logged in to the system. The user has draft access.
Post-condition	The draft is saved, updated, or deleted after publishing. A published draft becomes a normal post.

Table 3.38: Use Case Specification - Manage Drafts

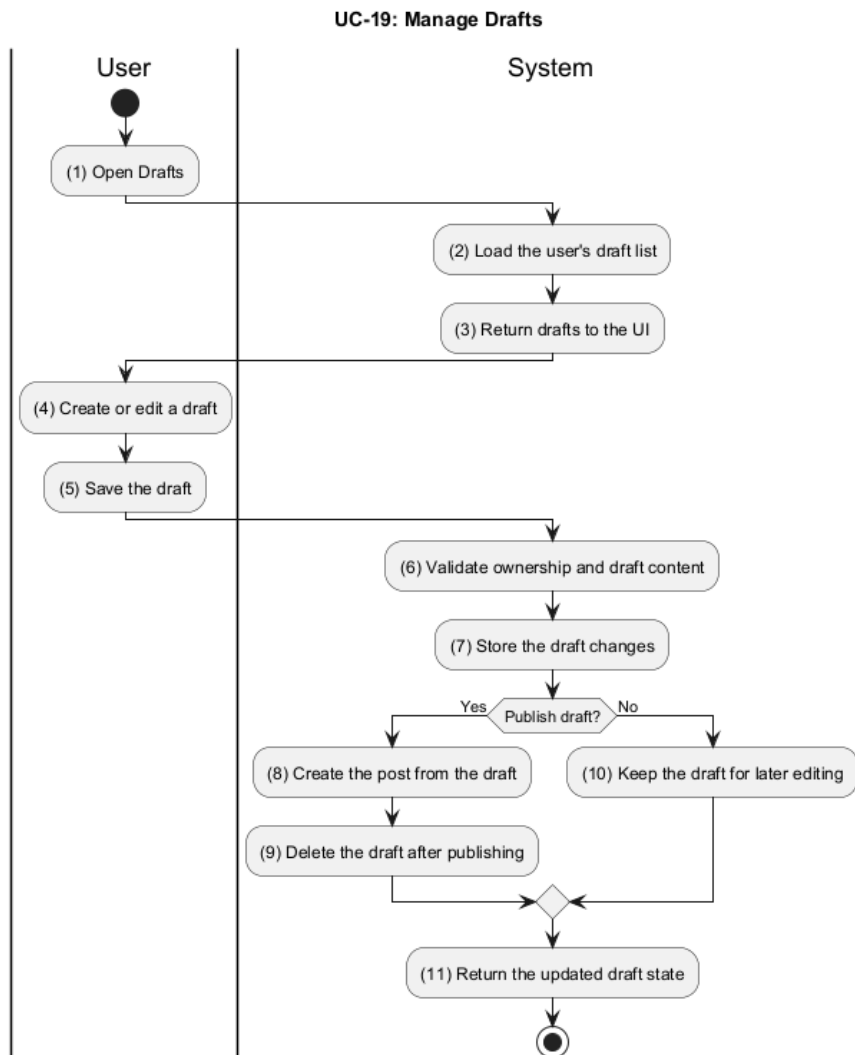


Figure 3.20: Activity Diagram for UC-19

ID	Activity	Rule Type	Description
BR-UC19-01	(2–3)	Workflow Rules	Load the User’s Draft List: - The service loads drafts by author using <i>GetDraftsByAuthor()</i>. - The service returns the draft summaries with pagination metadata.
BR-UC19-02	(6)	Authorization Rules	Validate Draft Ownership and Payload: - The controller requires an authenticated user before draft operations. - if <code>authUser == null</code>, then the controller returns a forbidden response. - The service parses the author ID and draft ID to ObjectIDs. - if either ID cannot be parsed, then the service returns an invalid-ID error. - The service loads the draft and if the draft author does not match the requester, then the service returns a forbidden error.
BR-UC19-03	(6–7)	Workflow Rules	Store Draft Changes: - The service creates or updates the draft record with community, type, title, text, tags, images, videos, and poll data. - The draft record updates UpdatedAt whenever changes are saved. - if the draft does not exist or the author is invalid, then the operation stops before saving.

Table 3.39: Business Rules for UC-19 - Manage Drafts

ID	Activ- ity	Rule Type	Description
BR-UC19-04	(8–10)	Workflow Rules	Publish or Keep the Draft: - if publish is requested, then the service validates that title, community, and type are present. - The service creates a post from the draft using <i>CreatePost()</i>. - After a successful publish, the service deletes the draft record. - if publish is not requested, then the service keeps the draft for later editing.
BR-UC19-05	(11)	Message Rules	Return Updated Draft State: The system returns the updated draft or published post response after the operation completes.
BR-UC19-06	(11)	Message Rules	Return Draft Error Message: if authentication, ownership, validation, or persistence fails, then the controller returns the corresponding error message.

Table 3.39: Business Rules for UC-19 - Manage Drafts

3.1.20 UC-20 - Private Messaging

Name	Private Messaging
Description	This use case describes how users exchange instant one-to-one messages through the messaging system.
Actor	User
Trigger	When the user opens a conversation and sends a message.
Pre-condition	The user is logged in to the system. The conversation channel exists and is accessible.
Post-condition	The message is delivered to the conversation members. The channel read state is updated.

Table 3.40: Use Case Specification - Private Messaging

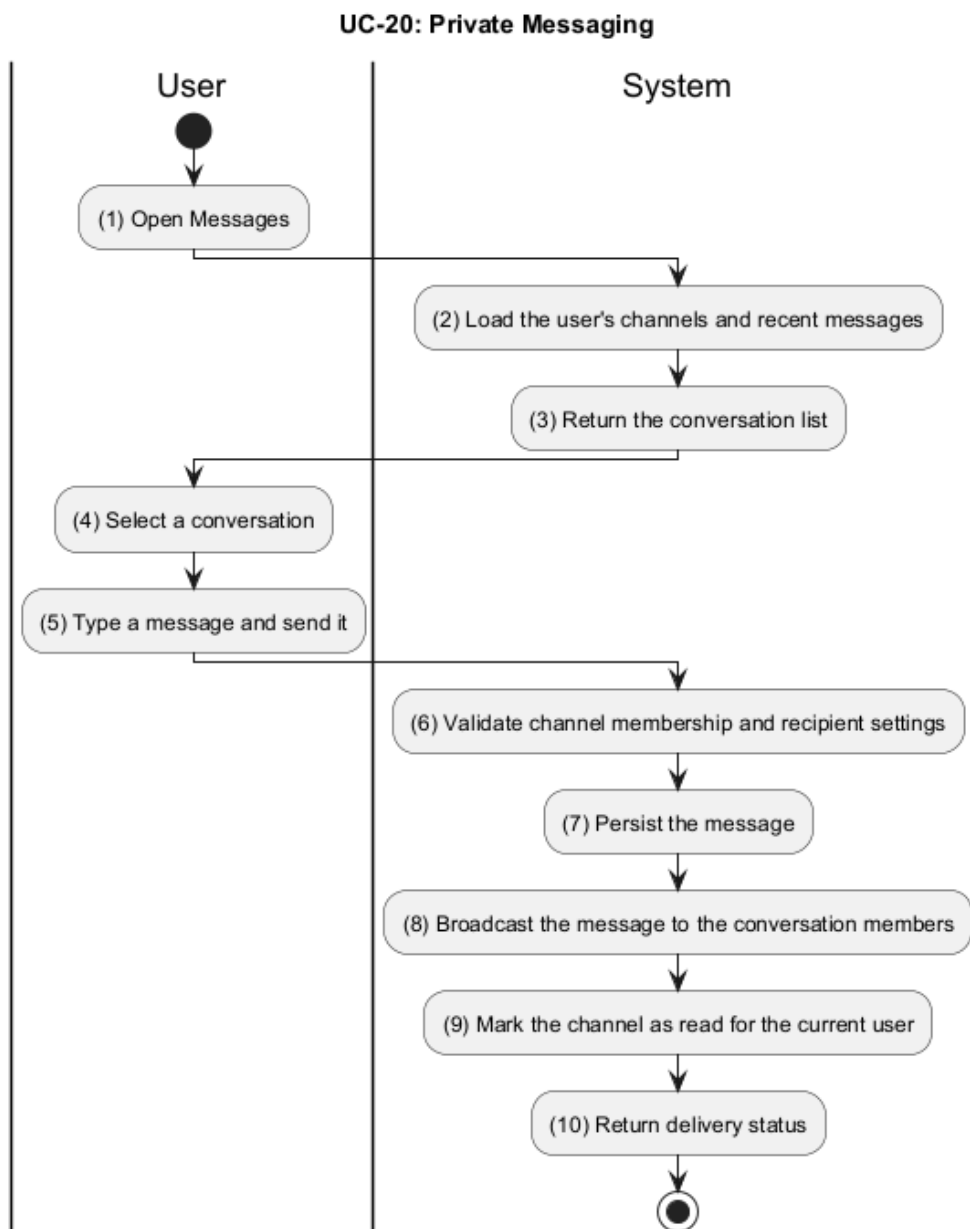


Figure 3.21: Activity Diagram for UC-20

ID	Activ- ity	Rule Type	Description
BR-UC20-01	(2)	Authorization Rules	Load the User's Conversations: - The controller requires an authenticated user before loading conversation data. - if authUser == null, then the controller returns a forbidden response. - The channel service only returns channels when the requester matches the target user ID. - if the requester is not the same as the target user, then the service returns a forbidden error.
BR-UC20-02	(2-3)	Workflow Rules	Return Conversation List and Recent Message State: - The service loads the user's channels using <i>GetChannelsByUserID()</i>. - The service returns the conversation list together with pagination data. - The message filter endpoint returns the user's messages only when the requester is a member of the channel.
BR-UC20-03	(6)	Validation Rules	Validate New Message Payload and Channel Access: - The websocket hub decodes the new_message payload before publishing the event. - if the payload is malformed or required fields are missing, then the hub sends an error message back to the sender. - The message service validates that temp_message_id, channel_id, sender_id, sender_username, and content are present. - The service loads the target channel and verifies that the sender is a member of that channel. - The service checks each recipient's direct-message setting before continuing.

Table 3.41: Business Rules for UC-20 - Private Messaging

ID	Activ- ity	Rule Type	Description
BR-UC20-04	(7-8)	Workflow Rules	Persist and Broadcast the Message: - The service creates a Message with IsSend = false, IsRead = false, and IsDeleted = false. - The repository stores the message and returns the created record. - The message service publishes a broadcast event so the websocket hub can send an ACK to the sender and a delivery event to the recipients. - if persistence fails, then the service publishes a message error event instead of broadcasting success.
BR-UC20-05	(9)	Workflow Rules	Mark the Channel as Read: - The message service marks the channel as read only for channel members. - The repository updates unread messages to is_read = true for messages not sent by the current user.
BR-UC20-06	(10)	Message Rules	Return Delivery Status: - The websocket hub sends an ACK back to the sender with the stored message payload. - The websocket hub broadcasts the created message to the other channel members.
BR-UC20-07	(10)	Message Rules	Return Messaging Error: if authentication, membership, recipient privacy, or persistence fails, then the system returns the corresponding error message to the sender.

Table 3.41: Business Rules for UC-20 - Private Messaging

3.1.21 UC-21 - Manage Notifications

Name	Manage Notifications
Description	This use case describes how a user views and clears system notifications.
Actor	User
Trigger	When the user opens the notifications page or marks notifications as read.
Pre-condition	The user is logged in to the system. Notifications exist for the account.
Post-condition	The selected notifications are marked as read or deleted. The notification list is refreshed.

Table 3.42: Use Case Specification - Manage Notifications

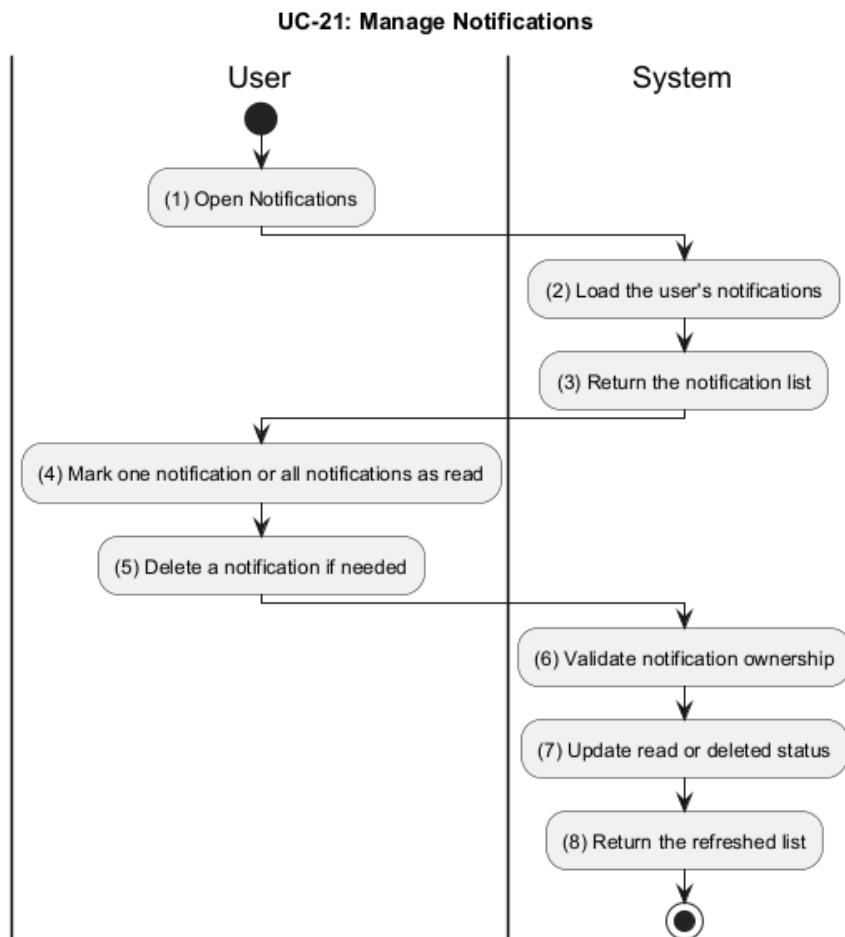


Figure 3.22: Activity Diagram for UC-21

ID	Activ- ity	Rule Type	Description
BR-UC21-01	(2–3)	Workflow Rules	Load Notifications: - The system loads notifications for recipientID using <i>GetNotifications()</i>. - The system returns the notification list with pagination data.
BR-UC21-02	(6)	Authorization Rules	Validate Notification Ownership: - The notification routes require authentication before any operation is processed. - The system only updates or deletes notifications that belong to the authenticated user. - If the notification ID is missing or the current user does not own the record, the operation is rejected.
BR-UC21-03	(6)	Validation Rules	Validate Notification ID for Single-Item Actions: - If the user marks one notification as read or deletes one notification, the system validates notificationID. - The service converts notificationID to an object ID before calling the repository.
BR-UC21-04	(7)	Workflow Rules	Update Notification State: - If the user marks all notifications as read, the system updates them using <i>MarkAllAsRead()</i>. - If the user marks a single notification as read, the system updates it using <i>MarkAsRead()</i>. - If the user deletes a notification, the system removes it using <i>DeleteNotification()</i>. - The repository filters updates by both notification ID and recipient ID.

Table 3.43: Business Rules for UC-21 - Manage Notifications

ID	Activ-ity	Rule Type	Description
BR-UC21-05	(8)	Message Rules	Return the Refreshed Notification List: The controller returns the updated list or success response after the operation completes.
BR-UC21-06	(8)	Message Rules	Return Notification Error Message: If authentication, ownership, validation, or persistence fails, the controller returns the corresponding error response.

Table 3.43: Business Rules for UC-21 - Manage Notifications

3.1.22 UC-22 - Report Violation

Name	Report Violation
Description	This use case describes how a user reports inappropriate content or behavior for review.
Actor	User
Trigger	When the user submits a report form.
Pre-condition	The user is logged in to the system. The reported target is available.
Post-condition	The report is stored in the moderation queue. The report can be reviewed by moderators or admins.

Table 3.44: Use Case Specification - Report Violation

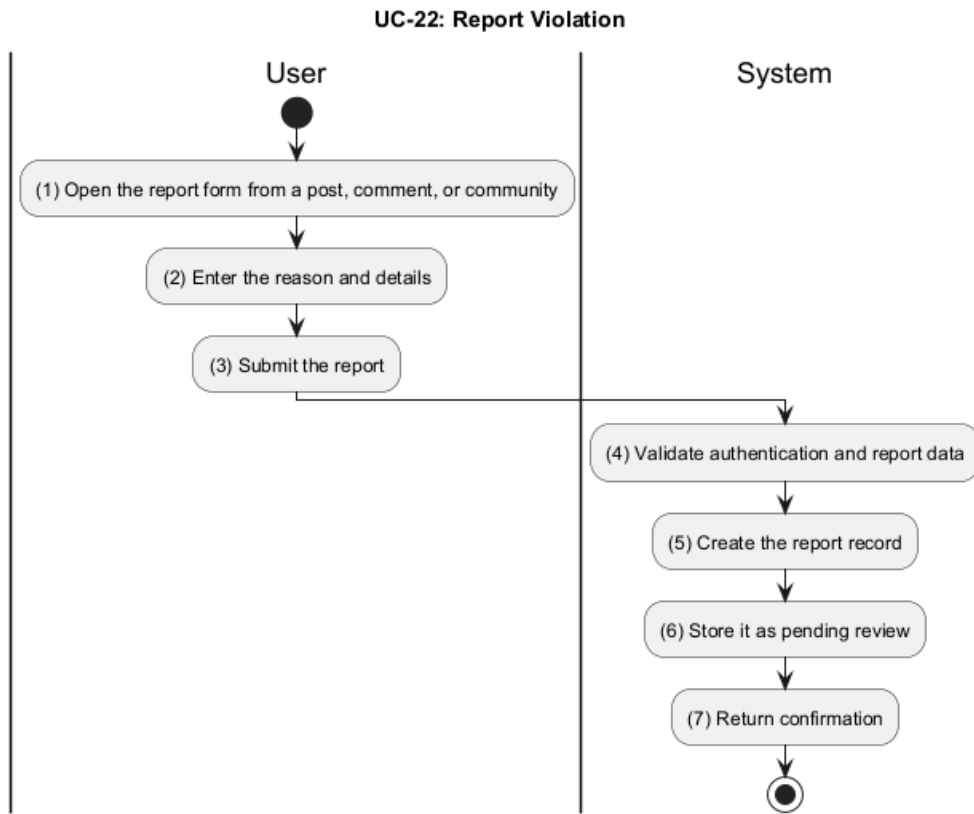


Figure 3.23: Activity Diagram for UC-22

ID	Activ-ity	Rule Type	Description
BR-UC22-01	(4)	Authorization / Validation Rules	Submit Report Request: - The report action requires an authenticated user. - The report form is opened from a post detail or post card action. - The controller binds the JSON body to <code>ReportPostRequest</code>. - The submitted reason is required. - The submitted description is optional. - if <code>authUser == null</code>, then return a forbidden response. - if the payload is invalid, then return a bad request response.

Table 3.45: Business Rules for UC-22 - Report Violation

ID	Activ- ity	Rule Type	Description
BR-UC22-02	(4)	Validation Rules	Validate the Target Post: - The service parses reporterID and postID to object IDs. - The service loads the target post before creating the report. - if the post does not exist or the IDs are invalid, then stop the operation and return the corresponding error.
BR-UC22-03	(4)	Constraint Rules	Prevent Invalid Reports: - The system rejects reports submitted by the author of the target post. - The system rejects duplicate reports from the same user for the same post.
BR-UC22-04	(5–6)	Workflow Rules	Create and Store the Report: - The service creates a Report with TargetType = post. - The repository stores the report with IsDeleted = false. - The report remains pending until reviewed by moderators or administrators.
BR-UC22-05	(7)	Message Rules	Return Operation Result: - if the report is successfully stored, then return a success response. - if authentication, validation, ownership, duplicate-report, or persistence checks fail, then return the corresponding error response.

Table 3.45: Business Rules for UC-22 - Report Violation

3.1.23 UC-23 - Create Community

Name	Create Community
Description	This use case describes how a user creates a new community and becomes its owner.
Actor	User
Trigger	When the user submits the create community form.
Pre-condition	The user is logged in to the system. The community name is available and valid.
Post-condition	The community is created successfully. The creator becomes the owner and first member.

Table 3.46: Use Case Specification - Create Community

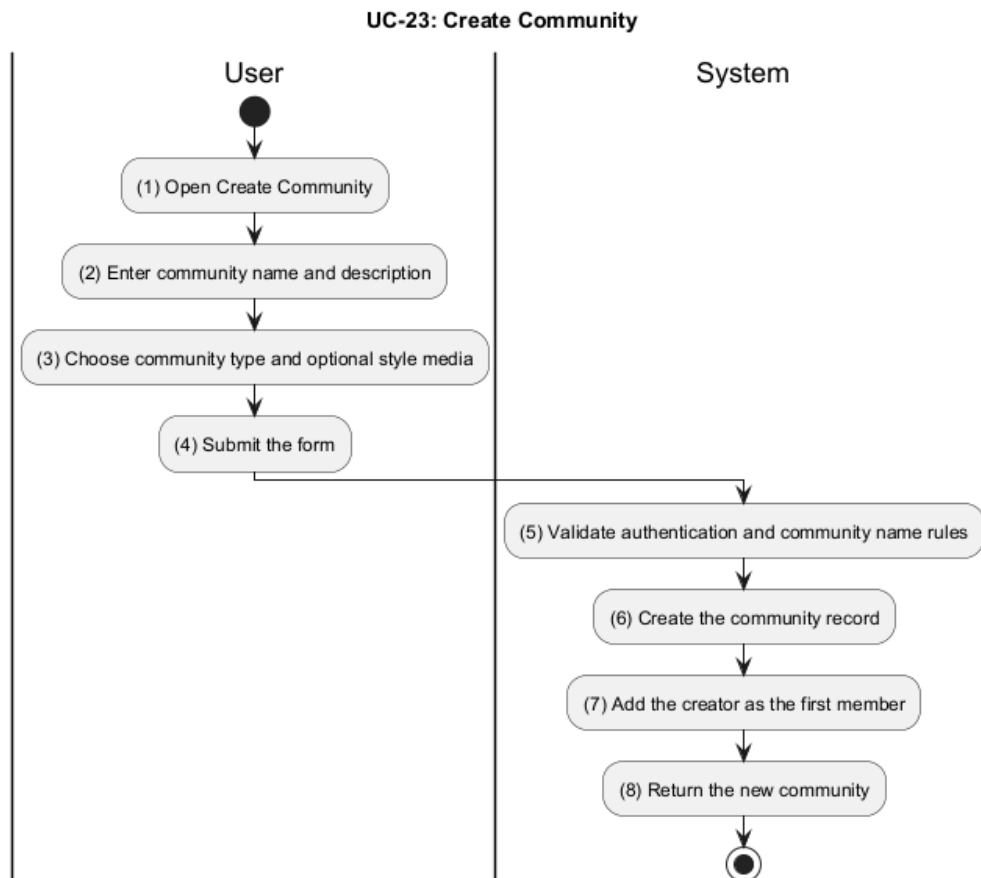


Figure 3.24: Activity Diagram for UC-23

ID	Activ- ity	Rule Type	Description
BR-UC23-01	(5)	Authorization Rules	Open and Submit the Create Community Form: - The create-community action requires an authenticated user. - if authUser == null, then the controller returns a forbidden response. - The controller passes the authenticated user's ID to the service when creating the community.
BR-UC23-02	(5)	Validation Rules	Validate Community Input: - The controller binds the request to CreateCommunityRequest. - The community name is required and must satisfy the configured length rule. - The community description, when provided, must respect its length limit. - The request may include optional avatar, banner, setting, rules, moderators, creator metadata, and adult-content flag.
BR-UC23-03	(5)	Constraint Rules	Prevent Duplicate Community Names: - The repository enforces a unique index on community name. - if the same community name already exists, then the service returns a community-name-exists error.
BR-UC23-04	(6)	Workflow Rules	Create the Community Record: - The service creates a community with the submitted fields and the creator ID. - The new community is stored with IsDeleted = false and IsBanned = false. - The service persists the community through CreateCommunity().

Table 3.47: Business Rules for UC-23 - Create Community

ID	Activ-ity	Rule Type	Description
BR-UC23-05	(7)	Workflow Rules	Create the Initial Membership: - After the community is created, the service creates a membership record for the creator. - The creator becomes the first member of the new community.
BR-UC23-06	(8)	Message Rules	Return the New Community: The controller returns the created community with a success response after persistence succeeds.
BR-UC23-07	(8)	Message Rules	Return Community Creation Error: if authentication, validation, duplicate-name, or persistence checks fail, then the controller returns the corresponding error response.

Table 3.47: Business Rules for UC-23 - Create Community

3.1.24 UC-24 - Approve / Reject Post

Name	Approve / Reject Post
Description	This use case describes how a moderator reviews pending or edited posts and decides whether to publish them.
Actor	Moderator
Trigger	When the moderator opens the moderation queue and chooses approve or reject.
Pre-condition	The moderator is logged in to the system. The target post is pending review or edited.
Post-condition	The post is approved or rejected. The moderation result is stored and reflected in the community.

Table 3.48: Use Case Specification - Approve / Reject Post

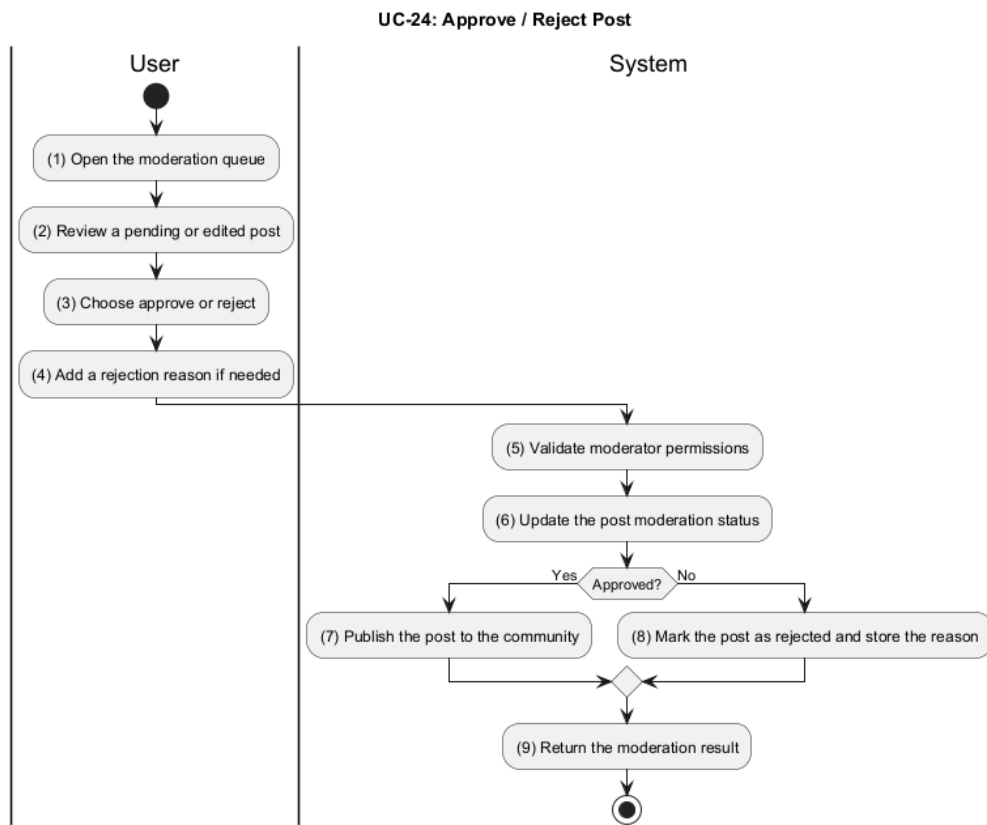


Figure 3.25: Activity Diagram for UC-24

ID	Activ-ity	Rule Type	Description
BR-UC24-01	(5)	Authorization Rules	Open the Moderation Queue and Validate Moderator Permissions: - The moderation queue endpoints require authentication. - The service checks that the requester is a moderator of the target community before loading or moderating posts. - if <code>authUser == null</code> or the user is not a moderator, then the system returns a forbidden response.

Table 3.49: Business Rules for UC-24 - Approve / Reject Post

ID	Activ- ity	Rule Type	Description
BR-UC24-02	(2)	Workflow Rules	Load Pending or Edited Posts: - The system loads pending posts with <i>GetPendingPosts()</i>. - The system loads edited posts with <i>GetEditedPosts()</i>. - The queued posts are filtered by community ID and moderation status before being returned.
BR-UC24-03	(5)	Validation Rules	Validate the Moderation Decision: - The controller binds the request to <code>ModeratePostRequest</code>. - The request must include the community ID, post ID, and approve flag. - if the moderation route parameters are missing or the request body is invalid, then the controller returns a bad request response.
BR-UC24-04	(5)	Validation Rules	Validate the Target Post: - The service loads the target post before applying the decision. - The post must belong to the same community as the moderation request. - The post must still be in moderation_status = pending. - if the post is missing, in another community, or not pending, then the operation stops with the corresponding error.
BR-UC24-05	(7)	Workflow Rules	Approve the Post: - if approve = true, then the service sets moderation_status = approved. - The service updates moderated_at for the post. - The service publishes a <code>PostApprovedEvent</code> so the post becomes visible to the community flow.

Table 3.49: Business Rules for UC-24 - Approve / Reject Post (Continued)

ID	Activ-ity	Rule Type	Description
BR-UC24-06	(8)	Workflow Rules	Reject the Post: - if approve = false, then the service sets moderation_status = rejected. - The service stores a ModerationResult with IsViolation = true and the rejection reason. - if no rejection reason is supplied, then the service uses the default moderator rejection reason. - The service publishes a PostRejectedEvent for downstream notifications.
BR-UC24-07	(9)	Message Rules	Return the Moderation Result: The controller returns a success response with the post ID and final status after the update succeeds.
BR-UC24-08	(9)	Message Rules	Return Moderation Error Message: if authentication, permission, validation, lookup, or persistence checks fail, then the controller returns the corresponding error response.

Table 3.49: Business Rules for UC-24 - Approve / Reject Post

3.1.25 UC-25 - Ban / Mute User

Name	Ban / Mute User
Description	This use case describes how a moderator restricts a user's participation in a community.
Actor	Moderator
Trigger	When the moderator chooses to ban or mute a user.
Pre-condition	The moderator is logged in to the system. The target user belongs to the community.
Post-condition	The ban or mute restriction is stored. The user is prevented from performing the restricted actions.

Table 3.50: Use Case Specification - Ban / Mute User

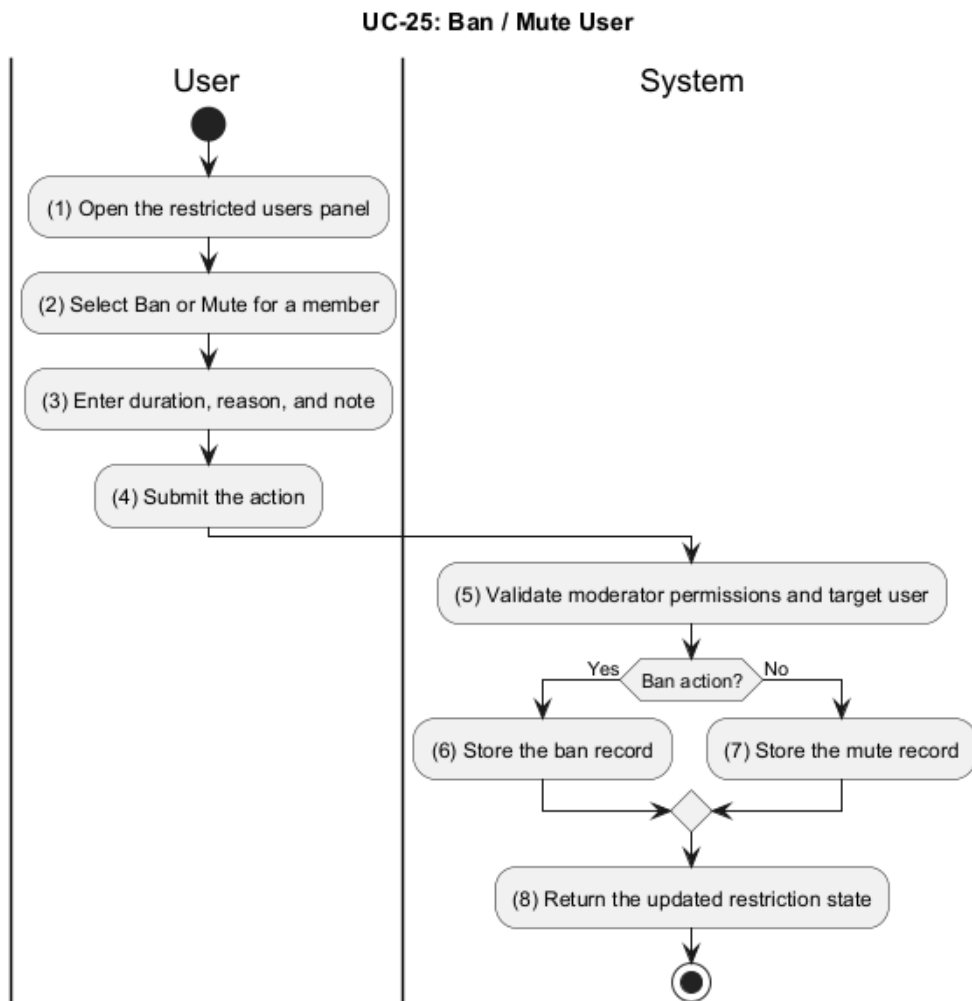


Figure 3.26: Activity Diagram for UC-25

ID	Activ-ity	Rule Type	Description
BR-UC25-01	(5)	Authorization Rules	Open the Restricted Users Panel and Validate Moderator Access: - The ban and mute actions require authentication. - The service checks that the requester is a moderator of the target community. - if authUser == null or the requester is not a moderator, then the system returns a forbidden response.

Table 3.51: Business Rules for UC-25 - Ban / Mute User

ID	Activ- ity	Rule Type	Description
BR-UC25-02	(5)	Validation Rules	Validate the Restriction Request: - The system binds the request to CommunityBanUserRequest. - The request must include communityID, userID, type, and lengthDays. - The restriction type must be either banned or muted. - The reason field is accepted by the API and is stored with the restriction record.
BR-UC25-03	(5)	Validation Rules	Validate the Target User and Community: - The service converts the community ID, user ID, and requester ID to object IDs. - if any ID is invalid, then the operation stops with the corresponding error.
BR-UC25-04	(6)	Workflow Rules	Store the Ban Record: - if type = banned, then the service creates a CommunityBan record with Type = banned. - The service calculates ExpiresAt from lengthDays. - The repository clears any existing active restriction for the same user and community before inserting the new ban. - The ban record stores the target user, community, reason, requester, banned time, expiration time, and soft-delete flags.

Table 3.51: Business Rules for UC-25 - Ban / Mute User (Continued)

ID	Activ-ity	Rule Type	Description
BR-UC25-05	(7)	Workflow Rules	Store the Mute Record: - if type = muted, then the service creates a CommunityBan record with Type = muted. - The same expiration and audit fields are stored for mute actions. - The repository clears any existing active restriction for the same user and community before inserting the new mute.
BR-UC25-06	(8)	Message Rules	Return the Updated Restriction State: The controller returns a success response after the restriction is saved.
BR-UC25-07	(8)	Message Rules	Return Restriction Error Message: if authentication, permission, validation, ID conversion, or persistence checks fail, then the controller returns the corresponding error response.

Table 3.51: Business Rules for UC-25 - Ban / Mute User

3.1.26 UC-26 - Configure Community

Name	Configure Community
Description	This use case describes how a community owner updates rules, privacy, and appearance settings.
Actor	Owner
Trigger	When the owner opens the community settings page and saves changes.
Pre-condition	The owner is logged in to the system. The owner has permission to manage the community.
Post-condition	The community configuration is updated. The new settings are applied to the community.

Table 3.52: Use Case Specification - Configure Community

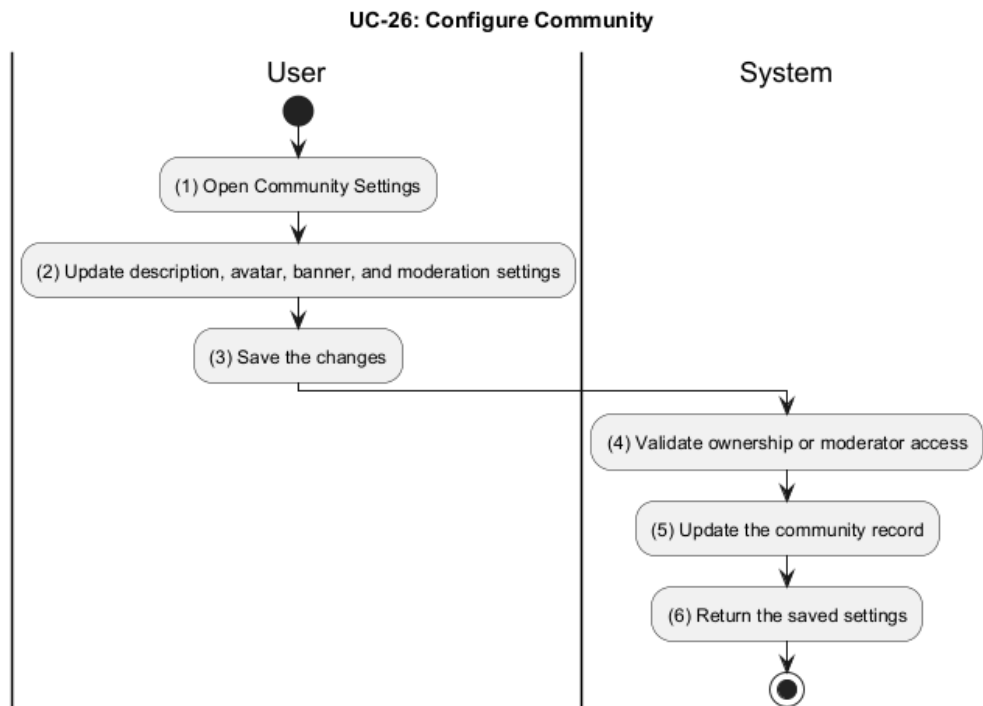


Figure 3.27: Activity Diagram for UC-26

ID	Activ-ity	Rule Type	Description
BR-UC26-01	(4)	Authorization Rules	Open Community Settings and Validate Access: - The community settings page requires a logged-in user. - The service allows the update only when the requester is the community creator or an active moderator. - if authUser == null or the requester lacks moderator access, then the system returns a forbidden response.

Table 3.53: Business Rules for UC-26 - Configure Community

ID	Activity	Rule Type	Description
BR-UC26-02	(4)	Validation Rules	Validate the Settings Request: - The controller binds the request to <code>UpdateCommunityRequest</code>. - The request must include the community ID. - The request may include description, avatar, banner, moderation settings, and rules. - if the payload cannot be bound, then the controller returns a bad request response.
BR-UC26-03	(5)	Workflow Rules	Load the Community Record: - The service loads the target community before applying changes. - if the community does not exist, then the operation stops with the corresponding error.
BR-UC26-04	(5)	Workflow Rules	Apply Community Field Updates: - The service updates only the non-nil fields from the request. - The configurable fields are description, avatar, banner, settings, and rules. - The service rejects save attempts that do not change any field.
BR-UC26-05	(5)	Constraint Rules	Require at Least One Change: if no update fields are provided, then the service returns <code>ErrNoFieldsToUpdate</code>.
BR-UC26-06	(5)	Workflow Rules	Persist the Community Settings: - The service saves the updated community through <i><code>UpdateCommunity()</code></i>. - The repository replaces the full community record after the in-memory updates are applied.

Table 3.53: Business Rules for UC-26 - Configure Community (Continued)

ID	Activ- ity	Rule Type	Description
BR-UC26-07	(6)	Message Rules	Return the Saved Community Settings: The controller returns the updated community after persistence succeeds.
BR-UC26-08	(6)	Message Rules	Return Community Update Error: if authentication, permission, validation, lookup, or persistence checks fail, then the controller returns the corresponding error response.

Table 3.53: Business Rules for UC-26 - Configure Community

3.1.27 UC-27 - Assign Roles

Name	Assign Roles
Description	This use case describes how a community owner appoints moderators or other roles for members.
Actor	Owner
Trigger	When the owner invites or activates a role for another member.
Pre-condition	The owner is logged in to the system. The target member belongs to the community.
Post-condition	The role assignment is stored successfully. The target member gains the new permissions.

Table 3.54: Use Case Specification - Assign Roles

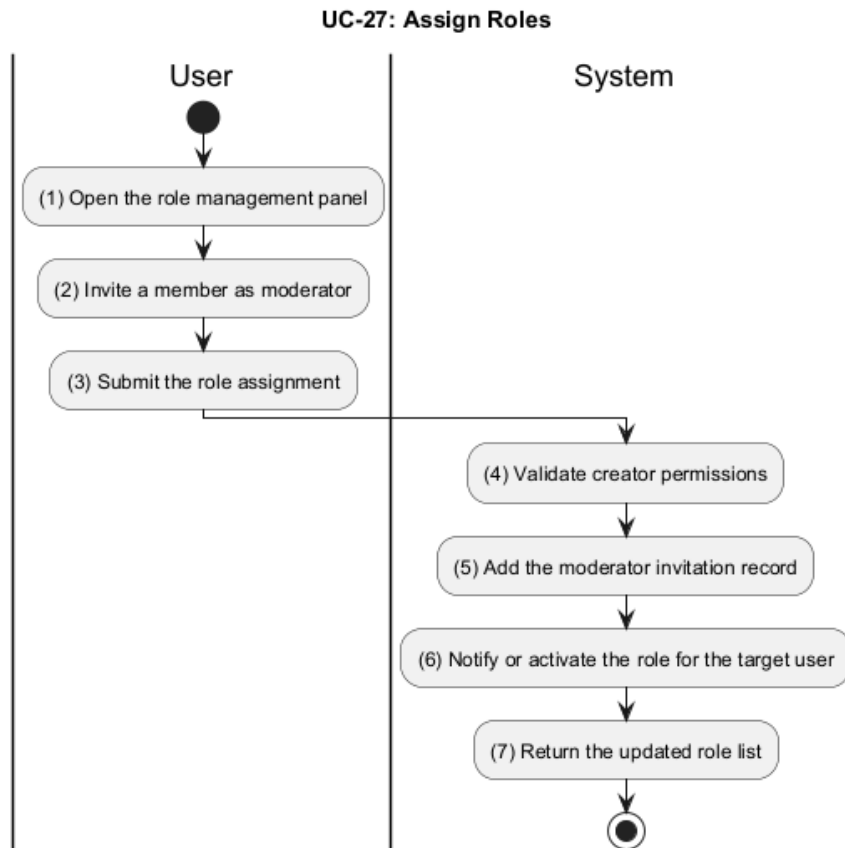


Figure 3.28: Activity Diagram for UC-27

ID	Activ-ity	Rule Type	Description
BR-UC27-01	(4)	Authorization Rules	Open the Role Management Panel and Validate Creator Access: - The role assignment actions require authentication. - The service allows adding moderators only when the requester is the community creator. - The target user must be invited from the creator's community role management flow. - if authUser == null or the requester is not the creator, then the system returns a forbidden response.

Table 3.55: Business Rules for UC-27 - Assign Roles

ID	Activ- ity	Rule Type	Description
BR-UC27-02	(4)	Validation Rules	Validate the Moderator Invitation Request: - The controller binds the request to <code>AddModeratorRequest</code>. - The request must include the community ID and at least one moderator user ID. - The target user IDs are resolved from the invited usernames in the current UI flow. - if the request payload cannot be bound or contains no target users, then the controller returns a bad request response.
BR-UC27-03	(5)	Constraint Rules	Prevent Duplicate Active Moderators: - The service rejects a user who is already an active moderator. - Existing inactive invitations for the same user are removed before a new invite is stored. - if the moderator already exists as active, then the service returns <code>ErrModeratorAlreadyExists</code>.
BR-UC27-04	(5)	Workflow Rules	Add the Moderator Invitation Record: - The service loads the community and the invited user before storing the role change. - The service creates a <code>Moderator</code> entry with <code>IsActive = false</code> and the current assignment time. - The community record is replaced after the invitation is appended. - The service publishes a <code>ModeratorAddedEvent</code> after persistence succeeds.

Table 3.55: Business Rules for UC-27 - Assign Roles (Continued)

ID	Activ-ity	Rule Type	Description
BR-UC27-05	(6)	Workflow Rules	Activate the Moderator Role: - The invited user can activate the moderator role through the activation endpoint. - The repository marks the matching moderator entry as active and refreshes the assignment timestamp. - if the invitation cannot be matched, then the repository returns a bad request error.
BR-UC27-06	(7)	Message Rules	Return the Updated Role List: The controller returns the updated role state after the invitation or activation succeeds.
BR-UC27-07	(7)	Message Rules	Return Role Assignment Error: if authentication, permission, validation, duplicate-role, lookup, or persistence checks fail, then the controller returns the corresponding error response.

Table 3.55: Business Rules for UC-27 - Assign Roles

3.1.28 UC-28 - Handle Global Reports

Name	Handle Global Reports
Description	This use case describes how a system admin reviews and acts on platform-wide violation reports.
Actor	System Admin
Trigger	When the admin opens the global reports dashboard.
Pre-condition	The admin is logged in to the system. There are reports available for review.
Post-condition	The selected reports are reviewed or deleted. The report queue is updated.

Table 3.56: Use Case Specification - Handle Global Reports

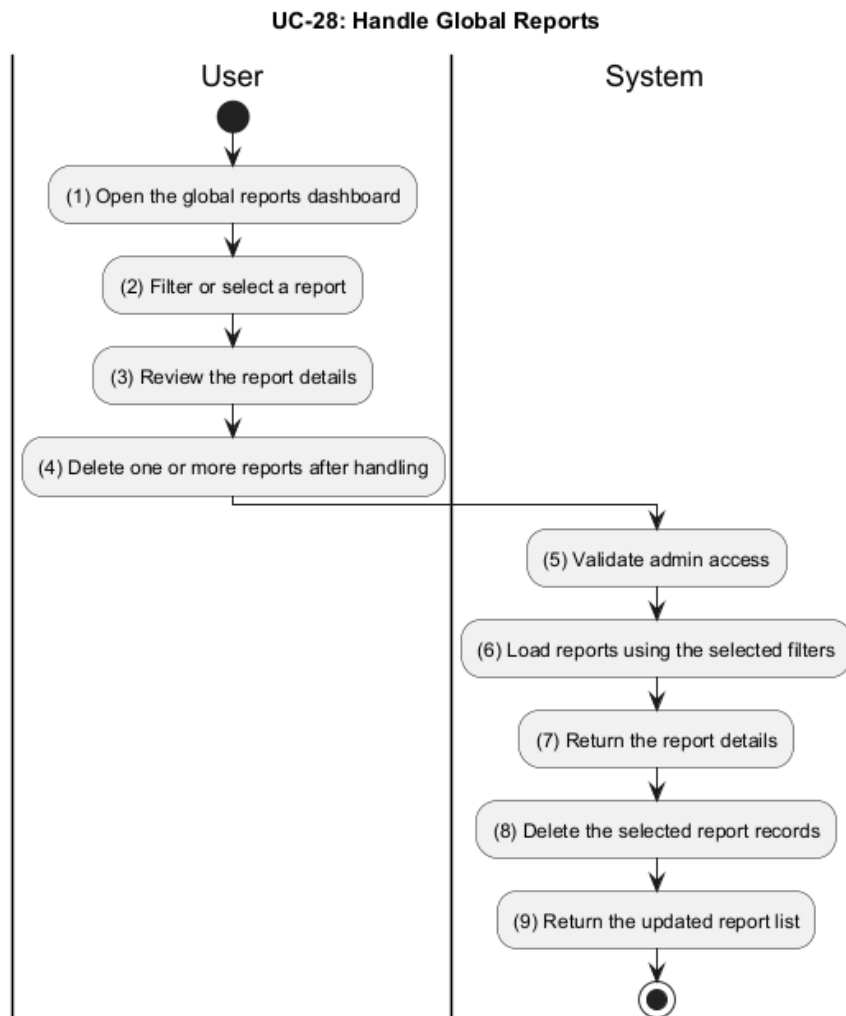


Figure 3.29: Activity Diagram for UC-28

ID	Activ- ity	Rule Type	Description
BR-UC28-01	(5)	Authorization Rules	Open the Global Reports Dashboard and Validate Admin Access: - The global reports routes require authentication and admin privileges. - if the requester is not an admin, then the system returns a forbidden response. - The dashboard and report detail actions are only available through the admin reports routes.

Table 3.57: Business Rules for UC-28 - Handle Global Reports

ID	Activ- ity	Rule Type	Description
BR-UC28-02	(6)	Validation Rules	Validate the Report Filter Request: - The system binds the query to GetReportsFilterQuery. - The request may include reporter ID, target ID, target type, reason, date range, page, and page size. - if the query cannot be bound, then the controller returns a bad request response.
BR-UC28-03	(7)	Workflow Rules	Load Report Details and Filtered Results: - The service loads reports using GetReportsFilter(). - The admin can open a single report using GetReportByIDAdmin().
BR-UC28-04	(8)	Validation Rules	Validate Selected Report IDs for Deletion: - The controller binds batch deletion requests to DeleteReportsRequest. - The request must contain at least one report ID. - if the batch list is empty or the request is invalid, then the controller returns a bad request response.
BR-UC28-05	(8)	Workflow Rules	Delete Single or Multiple Reports: - The admin can delete one report using DeleteReportByIDAdmin(). - The admin can delete multiple reports using DeleteReportsByIDAdmin(). - The service deletes the selected report records after validation succeeds.
BR-UC28-06	(9)	Message Rules	Return the Updated Report List: The controller returns the updated report state or success response after the selected reports are deleted.

Table 3.57: Business Rules for UC-28 - Handle Global Reports (Continued)

ID	Activ-ity	Rule Type	Description
BR-UC28-07	(9)	Message Rules	Return Report Management Error: if authentication, permission, validation, lookup, or persistence checks fail, then the controller returns the corresponding error response.

Table 3.57: Business Rules for UC-28 - Handle Global Reports

3.1.29 UC-29 - Manage Platform

Name	Manage Platform
Description	This use case describes how a system admin manages users and communities globally.
Actor	System Admin
Trigger	When the admin opens the platform management area and selects a target entity.
Pre-condition	The admin is logged in to the system. The admin has global management privileges.
Post-condition	The selected user or community is banned or unbanned. The platform state is updated accordingly.

Table 3.58: Use Case Specification - Manage Platform

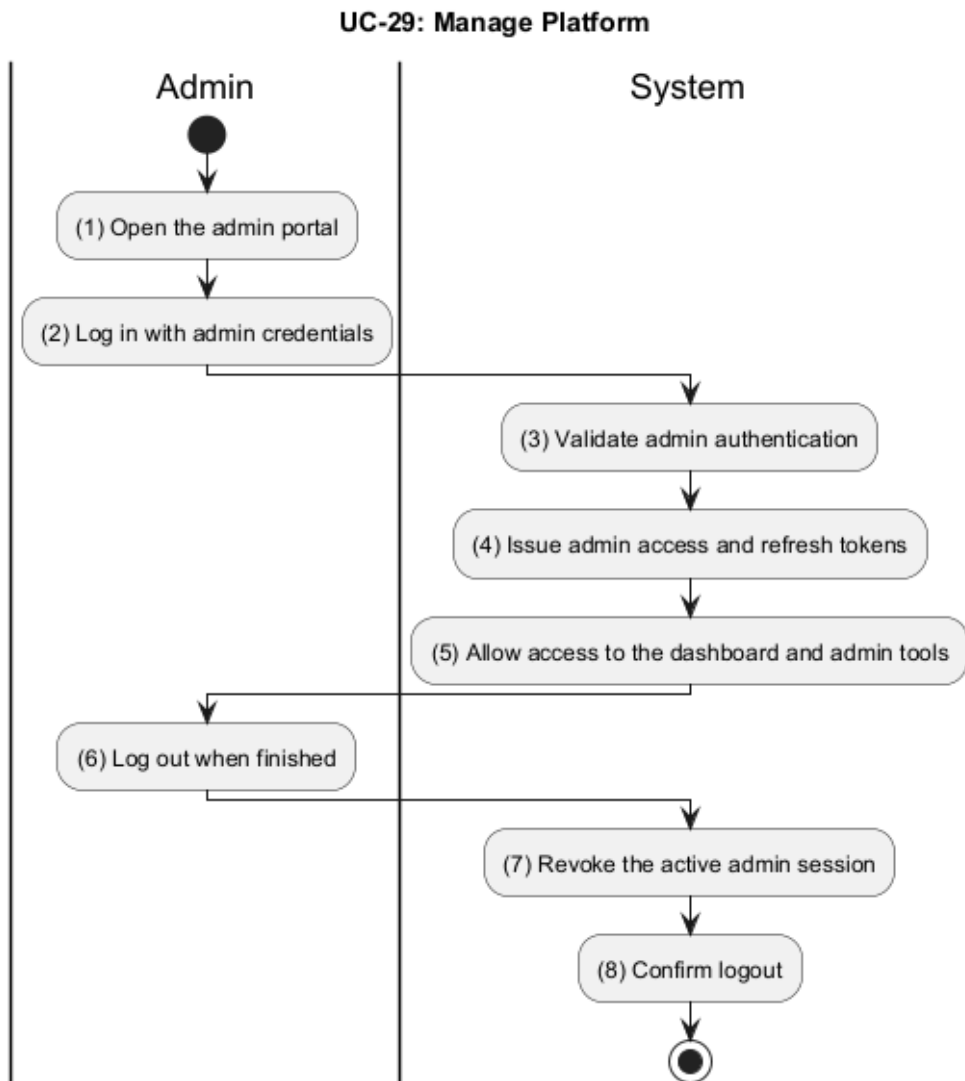


Figure 3.30: Activity Diagram for UC-29

ID	Activ-ity	Rule Type	Description
BR-UC29-01	(3)	Authorization Rules	Validate Admin Login and Access Rights: - The login endpoint accepts an identifier and password. - The service loads the account by email or username and requires the admin role. - The service rejects banned, deleted, or non-admin accounts before issuing tokens. - The service verifies the password for local accounts in <i>AdminLogin()</i>.
BR-UC29-02	(3)	Validation Rules	Validate the Admin Login Request: - The system binds the login payload to UserLoginRequest. - The request must include both identifier and password fields. - if the payload cannot be bound, then the system returns a bad request response. - if the credentials are invalid, then the service returns an invalid credentials response.
BR-UC29-03	(3-5)	Workflow Rules	Issue Admin Tokens and Allow Protected Access: - The service generates access and refresh tokens with <i>AdminLogin()</i> after successful authentication. - The login system returns the authenticated user together with both tokens. - Admin dashboard and tool routes require RequireAuth() and RequireAdmin() before they can be reached. - The protected admin stats routes expose the dashboard overview and related admin tools only after the token is accepted.

Table 3.59: Business Rules for UC-29 - Manage Platform

ID	Activ-ity	Rule Type	Description
BR-UC29-04	(7)	Workflow Rules	Revoke the Active Admin Session: - The logout endpoint reads the bearer access token and optional refresh token from the request. - The service extracts the token JTI's and invalidates them when token revocation is available in <i>AdminLogout()</i>. - The logout call completes even when token invalidation is unavailable, so the admin session ends cleanly.
BR-UC29-05	(8)	Message Rules	Return the Admin Auth Result: - The login response returns the user profile, access token, and refresh token. - The logout response confirms that the admin session has ended. - if authentication, validation, lookup, role, status, or token cleanup checks fail, then the system returns the corresponding error response.

Table 3.59: Business Rules for UC-29 - Manage Platform

3.1.30 UC-30 - View Analytics

Name	View Analytics
Description	This use case describes how a system admin reviews platform usage and performance metrics.
Actor	System Admin
Trigger	When the admin opens the analytics dashboard.
Pre-condition	The admin is logged in to the system. Analytics data is available for the selected period.
Post-condition	The analytics dashboard is displayed with current metrics. The admin can review user, content, and moderation statistics.

Table 3.60: Use Case Specification - View Analytics

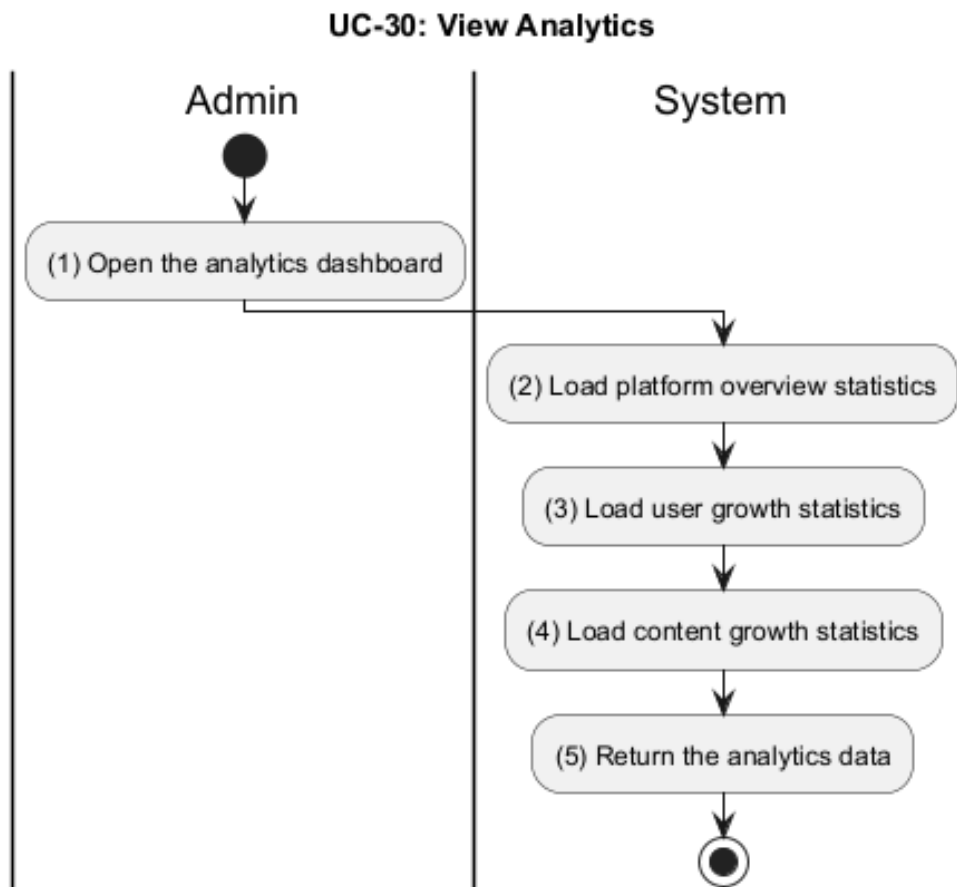


Figure 3.31: Activity Diagram for UC-30

ID	Activ-ity	Rule Type	Description
BR-UC30-01	(2)	Authorization Rules	Open the Analytics Dashboard with Admin Access: - The analytics routes require authentication and the <code>admin</code> role. - The protected admin stats routes are guarded by <code>RequireAuth()</code> and <code>RequireAdmin()</code>. - if the requester is not an admin, then the system returns a forbidden response.

Table 3.61: Business Rules for UC-30 - View Analytics

ID	Activ- ity	Rule Type	Description
BR-UC30-02	(2)	Validation Rules	Validate the Analytics Query Parameters: - The controller binds the user statistics query to <code>GetUserStatsQuery</code>. - The controller binds the content statistics query to <code>GetContentStatsQuery</code>. - The <code>period</code> field is optional and defaults to <code>week</code> when omitted. - if the query cannot be bound, then the controller returns a bad request response.
BR-UC30-03	(2)	Workflow Rules	Load the Platform Overview Snapshot: - The service loads the platform overview using <i><code>GetPlatformOverview()</code></i>. - The overview aggregates user, community, content, and moderation statistics from the repositories. - The overview response includes the current snapshot timestamp in <code>UpdatedAt</code>.
BR-UC30-04	(3–5)	Workflow Rules	Load User and Content Growth Statistics: - The service loads user statistics using <i><code>GetUserStats()</code></i> and content statistics using <i><code>GetContentStats()</code></i>. - The service selects the growth window from the requested period and falls back to <code>week</code> when needed. - The response includes the overview and the corresponding growth series for the requested analytics view.

Table 3.61: Business Rules for UC-30 - View Analytics (Continued)

ID	Activ-ity	Rule Type	Description
BR-UC30-05	(5)	Message Rules	Return the Analytics Result or Error: - The controller returns the analytics payload after the overview or growth data is loaded successfully. - if authentication, authorization, validation, lookup, or persistence checks fail, then the controller returns the corresponding error response.

Table 3.61: Business Rules for UC-30 - View Analytics

3.2 Data Requirements

3.2.1 Logical Data Model

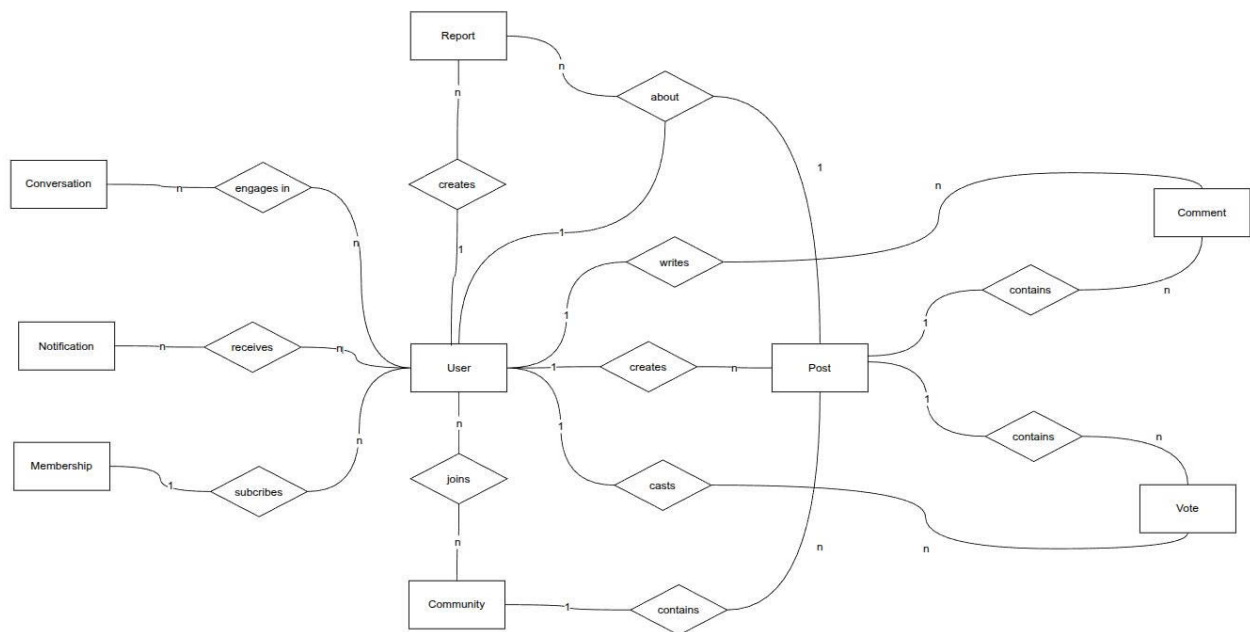


Figure 3.32: Logical Data Model for LKForum

3.2.2 Data Dictionary

Authentication and User Profile Models

Data Element	Description	Composition / Data Type	Length	Values
ID	Unique identifier of a user.	objectId	24	System-generated, unique.
Username	Display name used to identify a user account.	string	–	Must be unique within the platform.
Email	Email address used for authentication and contact.	string	–	Must be unique and valid.
Reputation	User reputation score.	integer	–	System-maintained counter.
Password	Hashed password for a local account.	string	–	Stored only for local providers.
Provider	Authentication source for a user account.	enum	–	local, google
ProviderID	External account identifier returned by the provider.	string	–	Used for third-party login linkage.
Role	Platform-wide role of the user.	enum	–	user, admin
RoleContent	Role-specific data attached to the user account.	object	–	Contains user or admin sub-document data.
Settings	User preference configuration.	object	–	Contains appearance, notification, privacy, and content settings.
IsVerified	Indicates whether the account has been verified.	boolean	–	true, false
IsBanned	Indicates whether the account is currently banned.	boolean	–	true, false
BanUntil	Timestamp when a ban expires, if any.	timestamp	–	Null means permanent ban.
BanReason	Reason recorded for a ban action.	string	–	Optional free text.
CreatedAt	Timestamp when the record was created.	timestamp	–	System-generated.
DeletedAt	Timestamp for soft-deleted records.	timestamp	–	Null when not deleted.

Table 3.62: User

Data Element	Description	Composition / Data Type	Length	Values
AsUser	User-specific profile data.	object	–	See UserRoleContent table.
AsAdmin	Admin-specific profile data.	object	–	See AdminRoleContent table.

Table 3.63: RoleContent

Data Element	Description	Composition / Data Type	Length	Values
Avatar	Profile image.	object	–	See Image table.
Cover	Cover image.	object	–	See Image table.
Bio	Short biography.	string	–	User-generated text.
Gender	Declared gender.	enum	–	male, female, prefer_not_to_say
DateOfBirth	User date of birth.	timestamp	–	Optional date value.
Location	Province or city.	enum	–	One of the provinces defined in model/location.go.
Interests	List of user interests.	array[string]	–	Values defined in model/interest.go.
SocialLinks	Collection of public social profiles.	object	–	Website, Facebook, YouTube, GitHub links.
Stats	User activity statistics.	object	–	Post, comment, and vote counters plus timestamps.

Table 3.64: UserRoleContent

Data Element	Description	Composition / Data Type	Length	Values
Website	Personal website URL.	string	–	Optional URL.
Facebook	Facebook profile URL.	string	–	Optional URL.
YouTube	YouTube channel URL.	string	–	Optional URL.
GitHub	GitHub profile URL.	string	–	Optional URL.

Table 3.65: SocialLinks

Data Element	Description	Composition / Data Type	Length	Values
PostCount	Number of posts created.	integer	–	System-maintained counter.
CommentCount	Number of comments created.	integer	–	System-maintained counter.
TotalUpvotes	Total upvotes received.	integer	–	System-maintained counter.
JoinedAt	Timestamp when the user joined.	timestamp	–	System-generated.
LastActiveAt	Timestamp of the last activity.	timestamp	–	System-generated.

Table 3.66: ActivityStats

Data Element	Description	Composition / Data Type	Length	Values
Permissions	Admin capability list.	array[string]	–	Permission names granted to the admin.

Table 3.67: AdminRoleContent

Data Element	Description	Composition / Data Type	Length	Values
Appearance	Visual preferences.	object	–	See AppearanceSettings table.
Notifications	Notification preferences.	object	–	See NotificationSettings table.
Privacy	Privacy preferences.	object	–	See PrivacySettings table.
Content	Content filtering preferences.	object	–	See ContentSettings table.

Table 3.68: UserSettings

Data Element	Description	Composition / Data Type	Length	Values
Theme	UI theme preference.	enum	–	light, dark, auto
FontSize	UI font size preference.	enum	–	small, medium, large

Table 3.69: AppearanceSettings

Data Element	Description	Composition / Data Type	Length	Values
InAppEnabled	In-app notification toggle.	boolean	–	true, false
EmailEnabled	Email notification toggle.	boolean	–	true, false
NotifyOn-Comment	Notify on comments.	boolean	–	true, false
NotifyOn-Mention	Notify on mentions.	boolean	–	true, false
NotifyOnUpvote	Notify on upvotes.	boolean	–	true, false
NotifyOnMessage	Notify on private messages.	boolean	–	true, false

Table 3.70: NotificationSettings

Data Element	Description	Composition / Data Type	Length	Values
ShowProfile	Controls whether a profile is public.	boolean	–	true, false
ShowEmail	Controls whether email is visible on the profile.	boolean	–	true, false
ShowPostHistory	Controls whether post history is visible.	boolean	–	true, false
AllowDirect-Messages	Controls whether direct messages are allowed.	boolean	–	true, false
AllowMentions	Controls whether mentions are allowed.	boolean	–	true, false

Table 3.71: PrivacySettings

Data Element	Description	Composition / Data Type	Length	Values
AllowNSFW	Controls whether NSFW content is allowed.	boolean	–	true, false

Table 3.72: ContentSettings

Data Element	Description	Composition / Data Type	Length	Values
EmailVerification-ID	Unique identifier of an email verification record.	objectId	24	System-generated, unique.

Table 3.73: Verification Records

EmailVerification- .Email	Email being verified.	string	–	Must be unique and valid.
EmailVerification- .OTP	One-time password used for verification.	string	6	Six-digit numeric code.
EmailVerification- .OTPExpiresAt	Expiration timestamp for the OTP.	timestamp	–	System-generated.
EmailVerification- .IsVerified	Indicates whether the OTP has already been verified.	boolean	–	true, false
EmailVerification- .Nonce	Anti-replay token.	string	–	System-generated random value.
EmailVerification- .CreatedAt	Timestamp when the record was created.	timestamp	–	System-generated.

Table 3.73: Verification Records

Data Element	Description	Composition / Data Type	Length	Values
PasswordReset- .ID	Unique identifier of a password reset record.	objectId	24	System-generated, unique.
PasswordReset- .Email	Email being reset.	string	–	Must be unique and valid.
PasswordReset- .OTP	One-time password used for reset.	string	6	Six-digit numeric code.
PasswordReset- .OTPExpiresAt	Expiration timestamp for the OTP.	timestamp	–	System-generated.
PasswordReset- .IsVerified	Indicates whether the OTP has already been verified.	boolean	–	true, false
PasswordReset- .Nonce	Anti-replay token.	string	–	System-generated random value.
PasswordReset- .CreatedAt	Timestamp when the record was created.	timestamp	–	System-generated.

Table 3.74: PasswordReset

Data Element	Description	Composition / Data Type	Length	Values
URL	Public URL of a stored image.	string	–	Cloudinary or equivalent media URL.

Table 3.75: Image

PublicID	Provider-specific image identifier.	string	–	Used for delete/update operations.
UploadedAt	Timestamp when the image was uploaded.	timestamp	–	System-generated.

Table 3.75: Image

Data Element	Description	Composition / Data Type	Length	Values
URL	Public URL of a stored video.	string	–	Cloudinary or equivalent media URL.
PublicID	Provider-specific video identifier.	string	–	Used for delete/update operations.
UploadedAt	Timestamp when the video was uploaded.	timestamp	–	System-generated.

Table 3.76: Video

Community and Membership Models

Data Element	Description	Composition / Data Type	Length	Values
ID	Unique identifier of a community.	objectId	24	System-generated, unique.
Name	Public name of a community.	string	–	Must be unique.
Description	Text describing a community.	string	–	Optional free text.
Avatar	Community avatar image URL.	string	–	Optional media URL.
Banner	Banner image for a community.	string	–	Optional media URL.
Setting	Community configuration settings.	object	–	Contains privacy and moderation flags.
Rules	Published rules of a community.	array[object]	–	Title and description entries.
Moderators	List of assigned community moderators.	array[object]	–	Moderator sub-documents.
MemberCount	Current number of community members.	integer	–	System-maintained counter.

Table 3.77: Community

PostCount	Current number of community posts.	integer	–	System-maintained counter.
CreateAt	Timestamp when the community was created.	timestamp	–	System-generated.
CreateByID	Identifier of the creator.	objectId	24	Must correspond to an existing user id.
CreateByName	Username of the creator.	string	–	Copied from the creator account.
CreateByAvatar	Avatar URL of the creator.	string	–	Copied from the creator profile.
Is18Plus	Marks the community as adult content.	boolean	–	true, false
IsDeleted	Indicates whether the community is soft deleted.	boolean	–	true, false
IsBanned	Indicates whether the community is banned.	boolean	–	true, false
BanReason	Reason recorded for a community ban.	string	–	Optional free text.

Table 3.77: Community

Data Element	Description	Composition / Data Type	Length	Values
IsPrivate	Controls whether the community is visible only to members.	boolean	–	true, false
PostRequire-Approval	Controls whether new posts require moderator approval.	boolean	–	true, false
JoinRequire-Approval	Controls whether join requests require approval.	boolean	–	true, false
MaxPostLength	Maximum allowed post length in characters.	integer	–	Positive integer.

Table 3.78: CommunitySetting

Data Element	Description	Composition / Data Type	Length	Values
Title	Short title for a community rule.	string	–	User-defined label.

Table 3.79: CommunityRule

Description	Detailed explanation of a community rule.	string	–	User-defined text.
-------------	---	--------	---	--------------------

Table 3.79: CommunityRule

Data Element	Description	Composition / Data Type	Length	Values
UserID	Identifier of the moderator account.	objectId	24	Must correspond to an existing user id.
Username	Moderator username.	string	–	Copied from the user account.
Avatar	Moderator avatar.	object	–	See Image table.
IsActive	Indicates whether the moderator role is active.	boolean	–	true, false
AssignedAt	Timestamp when the moderator role was assigned.	timestamp	–	System-generated.

Table 3.80: Moderator

Data Element	Description	Composition / Data Type	Length	Values
ID	Unique identifier of a membership record.	objectId	24	System-generated, unique.
UserID	Identifier of the member user.	objectId	24	Must correspond to an existing user id.
CommunityID	Identifier of the target community.	objectId	24	Must correspond to an existing community id.
Status	Membership review status.	enum	–	pending, approved, rejected
CreatedAt	Timestamp when the membership was created.	timestamp	–	System-generated.
User	Cached user summary for a membership entry.	object	–	Contains id, username, and avatar.

Table 3.81: Membership

Data Element	Description	Composition / Data Type	Length	Values
--------------	-------------	-------------------------	--------	--------

Table 3.82: UserInfo

ID	Identifier of the membership user summary.	objectId	24	Must correspond to an existing user id.
Username	Username of the membership user summary.	string	–	Copied from the user account.
Avatar	Avatar of the membership user summary.	object	–	See Image table.

Table 3.82: UserInfo

Data Element	Description	Composition / Data Type	Length	Values
CommunityID	Identifier of the banned community.	objectId	24	Must correspond to an existing community id.
UserID	Identifier of the banned user.	objectId	24	Must correspond to an existing user id.
Type	Ban category applied by the community.	enum	–	banned, muted
Reason	Reason recorded for the ban or mute.	string	–	Optional free text.
BannedAt	Timestamp when the restriction was applied.	timestamp	–	System-generated.
BannedBy	Identifier of the moderator who applied the restriction.	objectId	24	Must correspond to an existing user id.
ExpiresAt	Timestamp when the restriction expires.	timestamp	–	Null for permanent restrictions.

Table 3.83: CommunityBan

Content and Moderation Models

Data Element	Description	Composition / Data Type	Length	Values
ID	Unique identifier of a post.	objectId	24	System-generated, unique.
AuthorID	Identifier of the post author.	objectId	24	Must correspond to an existing user id.
CommunityID	Identifier of the community that owns the post.	objectId	24	Must correspond to an existing community id.

Table 3.84: Post

Type	Type of post content.	enum	–	text, poll, video, image
Title	Title of a post.	string	–	Text entered by the author.
Content	Main post payload.	object	–	Contains text, images, videos, or poll data.
VotesCount	Vote summary for a post or comment.	object	–	Up and down counts.
CommentsCount	Number of comments on a post.	integer	–	System-maintained counter.
HotScore	Ranking score used for best sorting.	float	–	Calculated from votes and time decay.
CreatedAt	Timestamp when the post was created.	timestamp	–	System-generated.
UpdatedAt	Timestamp of the last content update.	timestamp	–	Null until updated.
IsDeleted	Indicates whether the record is soft deleted.	boolean	–	true, false
IsHidden	Indicates whether the post is hidden by its author.	boolean	–	true, false
IsEdited	Indicates whether the post has been edited.	boolean	–	true, false
Tags	Tags attached to a post.	array[string]	–	Free-form keywords.
IsDraft	Indicates whether the post is saved as a draft.	boolean	–	true, false
IsBan	Indicates whether a post is banned.	boolean	–	true, false
BanReason	Reason recorded for a post ban.	string	–	Optional free text.
ModerationStatus	Current AI or moderator moderation state.	enum	–	pending, approved, rejected, skipped
ModerationResult	Moderation analysis details.	object	–	Contains violation flag, confidence, categories, and reason.
ModeratedAt	Timestamp when moderation happened.	timestamp	–	Null until moderated.

Table 3.84: Post

Data Element	Description	Composition / Data Type	Length	Values
--------------	-------------	-------------------------	--------	--------

Table 3.85: PostContent

Text	Text body of a post.	string	–	Optional for non-text posts.
Images	Attached images in a post.	array[object]	–	Array of image metadata.
Videos	Attached videos in a post.	array[object]	–	Array of video metadata.
Poll	Poll payload attached to a post.	object	–	Question, options, votes, expiry, and multi-select flag.

Table 3.85: PostContent

Data Element	Description	Composition / Data Type	Length	Values
Question	Question presented to poll voters.	string	–	Poll text.
Options	Available poll options.	array[object]	–	Each option has id, text, and votes.
TotalVotes	Total number of poll votes.	integer	–	System-maintained counter.
ExpiresAt	Poll expiry timestamp.	timestamp	–	Null when the poll never expires.
AllowMultiple	Controls whether voters may choose more than one option.	boolean	–	true, false

Table 3.86: Poll

Data Element	Description	Composition / Data Type	Length	Values
ID	Identifier of a poll option.	string	–	Unique within the poll.
Text	Display text of a poll option.	string	–	Option label.
Votes	Vote count for a poll option.	integer	–	System-maintained counter.

Table 3.87: PollOption

Data Element	Description	Composition / Data Type	Length	Values
--------------	-------------	-------------------------	--------	--------

Table 3.88: VotesCount

Up	Number of upvotes.	integer	–	System-maintained counter.
Down	Number of downvotes.	integer	–	System-maintained counter.

Table 3.88: VotesCount

Data Element	Description	Composition / Data Type	Length	Values
IsViolation	Indicates whether the content violates policy.	boolean	–	true, false
Confidence	Confidence score returned by moderation.	float	–	0.0 to 1.0
Categories	Categories detected by moderation.	array[string]	–	Issue labels such as hate speech or violence.
Reason	Human-readable moderation explanation.	string	–	Localized explanation text.
CheckedText	Indicates whether text was analyzed.	boolean	–	true, false
CheckedMedia	Indicates whether media was analyzed.	boolean	–	true, false

Table 3.89: ModerationResult

Data Element	Description	Composition / Data Type	Length	Values
ID	Unique identifier of a comment.	objectId	24	System-generated, unique.
Author	Author details attached to a comment.	object	–	Contains id, username, and avatar.
PostID	Identifier of the parent post.	objectId	24	Must correspond to an existing post id.
ParentID	Identifier of the parent comment, if any.	objectId	24	Null for top-level comments.
Content	Comment body text.	string	–	User-generated text.
VotesCount	Vote summary for a comment.	object	–	Up and down counts.
CreatedAt	Timestamp when the comment was created.	timestamp	–	System-generated.
DeletedAt	Timestamp for soft-deleted comments.	timestamp	–	Null when not deleted.

Table 3.90: Comment

IsDeleted	Indicates whether a comment is soft deleted.	boolean	–	true, false
-----------	--	---------	---	-------------

Table 3.90: Comment

Data Element	Description	Composition / Data Type	Length	Values
ID	Identifier of the comment author.	objectId	24	Must correspond to an existing user id.
Username	Username of the comment author.	string	–	Copied from the user account.
Avatar	Avatar for the comment author.	object	–	Image metadata.

Table 3.91: CommentAuthor

Data Element	Description	Composition / Data Type	Length	Values
ID	Unique identifier of a draft.	objectId	24	System-generated, unique.
AuthorID	Identifier of the draft author.	objectId	24	Must correspond to an existing user id.
CommunityID	Identifier of the draft community, if any.	string	–	Optional community id string.
Type	Post type stored in a draft.	enum	–	text, poll, video, image
Title	Draft title.	string	–	Optional until publish time.
Content	Draft content payload.	object	–	Same shape as post content.
Tags	Tags saved with the draft.	array[string]	–	Optional free-form keywords.
CreatedAt	Timestamp when the draft was created.	timestamp	–	System-generated.
UpdatedAt	Timestamp when the draft was last updated.	timestamp	–	System-generated.

Table 3.92: Draft

Data Element	Description	Composition / Data Type	Length	Values
URL	Public URL of a stored image.	string	–	Cloudinary or equivalent media URL.
PublicID	Provider-specific image identifier.	string	–	Used for delete/update operations.
UploadedAt	Timestamp when the image was uploaded.	timestamp	–	System-generated.

Table 3.93: Image

Data Element	Description	Composition / Data Type	Length	Values
URL	Public URL of a stored video.	string	–	Cloudinary or equivalent media URL.
PublicID	Provider-specific video identifier.	string	–	Used for delete/update operations.
UploadedAt	Timestamp when the video was uploaded.	timestamp	–	System-generated.

Table 3.94: Video

Engagement and Activity Models

Data Element	Description	Composition / Data Type	Length	Values
ID	Unique identifier of a saved-post record.	objectId	24	System-generated, unique.
UserID	Identifier of the user who saved the post.	objectId	24	Must correspond to an existing user id.
PostID	Identifier of the saved post.	objectId	24	Must correspond to an existing post id.
SavedAt	Timestamp when a post was saved.	timestamp	–	System-generated.

Table 3.95: SavedPost

Data Element	Description	Composition / Data Type	Length	Values
ID	Unique identifier of a liked-post record.	objectId	24	System-generated, unique.

Table 3.96: LikedPost

UserID	Identifier of the user who liked the post.	objectId	24	Must correspond to an existing user id.
PostID	Identifier of the liked post.	objectId	24	Must correspond to an existing post id.
LikedAt	Timestamp when a post was liked.	timestamp	–	System-generated.

Table 3.96: LikedPost

Data Element	Description	Composition / Data Type	Length	Values
ID	Unique identifier of a post history record.	objectId	24	System-generated, unique.
UserID	Identifier of the user who viewed the post.	objectId	24	Must correspond to an existing user id.
PostID	Identifier of the viewed post.	objectId	24	Must correspond to an existing post id.
ViewedAt	Timestamp when a post was viewed.	timestamp	–	System-generated.

Table 3.97: PostHistory

Communication Models

Data Element	Description	Composition / Data Type	Length	Values
ID	Unique identifier of a notification.	objectId	24	System-generated, unique.
RecipientID	Identifier of the notification recipient.	objectId	24	Must correspond to an existing user id.
ActorID	Identifier of the user who triggered the notification.	objectId	24	Optional when the system is the actor.
Type	Type of notification.	enum	–	comment, like, follow, mention, new_message, system
Message	Notification text shown to the recipient.	string	–	System-generated or templated text.
Link	URL or route associated with the notification.	string	–	Navigation target.
IsRead	Indicates whether a notification has been read.	boolean	–	true, false

Table 3.98: Notification

Metadata	Additional notification payload.	Dictionary	–	Arbitrary key-value data.
CreatedAt	Timestamp when the notification was created.	timestamp	–	System-generated.

Table 3.98: Notification

Data Element	Description	Composition / Data Type	Length	Values
ID	Unique identifier of a private message channel.	objectId	24	System-generated, unique.
Members	Channel participants.	array[object]	–	Member summaries.
Settings	Per-user channel settings.	array[object]	–	Nickname and notification preferences.
Status	Channel lifecycle status.	enum	–	active, block
CreatedAt	Timestamp when the channel was created.	timestamp	–	System-generated.
UpdatedAt	Timestamp when the channel was last updated.	timestamp	–	System-generated.

Table 3.99: Channel

Data Element	Description	Composition / Data Type	Length	Values
UserID	Identifier of the user owning the channel settings.	objectId	24	Must correspond to a channel member.
Nickname	Custom nickname for the other user in the channel.	string	–	Optional display label.
Notification	Controls notification delivery for the channel.	boolean	–	true, false
TypingIndicator	Controls typing indicator visibility.	boolean	–	true, false
IsDeleted	Indicates whether the channel setting is hidden.	boolean	–	true, false

Table 3.100: ChannelSetting

Data Element	Description	Composition / Data Type	Length	Values
--------------	-------------	-------------------------	--------	--------

Table 3.101: ChannelMember

UserID	Identifier of the channel member.	objectId	24	Must correspond to an existing user id.
Username	Username of the channel member.	string	–	Copied from the user account.
Avatar	Avatar URL of the channel member.	string	–	Optional media URL.

Table 3.101: ChannelMember

Data Element	Description	Composition / Data Type	Length	Values
ID	Unique identifier of a private message.	objectId	24	System-generated, unique.
ChannelID	Identifier of the private message channel.	objectId	24	Must correspond to an existing channel id.
SenderID	Identifier of the message sender.	objectId	24	Null for system messages.
SenderUsername	Cached sender username.	string	–	Copied from the sender account.
Type	Message category.	enum	–	user, system
Content	Message body text.	string	–	User-generated or system-generated text.
IsSend	Indicates whether the message was delivered successfully.	boolean	–	true, false
IsDeleted	Indicates whether the message is soft deleted.	boolean	–	true, false
DeletedAt	Timestamp when the message was deleted.	timestamp	–	Null when not deleted.

Table 3.102: Message

Reporting and Administration Models

Data Element	Description	Composition / Data Type	Length	Values
ID	Unique identifier of a report.	objectId	24	System-generated, unique.
ReporterID	Identifier of the user who submitted the report.	objectId	24	Must correspond to an existing user id.
TargetID	Identifier of the reported user, post, or comment.	objectId	24	Must correspond to an existing target id.

Table 3.103: Report

TargetType	Type of reported target.	enum	–	user, post, comment
Reason	Short reason for a report or moderation action.	string	–	Free text provided by the user or moderator.
Description	Longer report details or explanation.	string	–	Optional free text.
IsDeleted	Indicates whether the report is soft deleted.	boolean	–	true, false
DeletedAt	Timestamp for deleted reports.	timestamp	–	Null when not deleted.
CreatedAt	Timestamp when the report was created.	timestamp	–	System-generated.

Table 3.103: Report

Enum and Value

Data Element	Description	Composition / Data Type	Length	Values
AuthProvider	User authentication source.	enum	–	local, google
Role	Platform-wide user role.	enum	–	user, admin
Gender	Declared gender.	enum	–	male, female, prefer_not_to_say
VNProvince	User location value.	enum	–	One of the provinces defined in model/location.go.
Interest	User interest label.	enum	–	Values defined in model/interest.go.
PostType	Post content type.	enum	–	text, poll, video, image
Moderation-Status	Moderation state.	enum	–	pending, approved, rejected, skipped
Membership-Status	Membership review state.	enum	–	pending, approved, rejected
Notification-Type	Notification type.	enum	–	comment, like, follow, mention, new_message, system
ChannelStatus	Channel lifecycle status.	enum	–	active, block
MessageType	Message category.	enum	–	user, system
ReportTarget-Type	Reported target type.	enum	–	user, post, comment
VoteType	Vote direction type.	enum	–	up, down

Table 3.104: Enum and Value Classes

VoteTargetType	Vote target type.	enum	–	post, comment
----------------	-------------------	------	---	---------------

Table 3.104: Enum and Value Classes

3.3 Data Integrity, Retention, and Disposal

- DI-1:** The system shall retain user-deleted content (such as posts and comments) in a deactivated state for 30 days. After this period, the content will be permanently purged from the system. This allows users a grace period to recover accidentally deleted items.
- DI-2:** When a user requests to delete their account, the account shall be immediately deactivated and scheduled for permanent deletion. All personally identifiable information and associated content will be permanently deleted or anonymized after 90 days, unless required to be kept for legal reasons.
- DI-3:** Moderation data, including content reports submitted by users and action logs taken by moderators, shall be retained for one year after the report is closed to monitor for patterns of abuse.
- DI-4:** Core content such as active posts and comments will be retained indefinitely as long as the author’s account and the community they belong to exist.

4 Other Nonfunctional Requirements

4.1 Performance Requirements

4.2 Security Requirements

4.3 Software Quality Attributes

5 Other Requirements

6 Appendix

6.1 Messages Codes

6.2 Error Codes

Code	Message
INVALID_CREDENTIALS	Email hoặc mật khẩu không đúng
INVALID_TOKEN	Token không hợp lệ hoặc đã hết hạn
INVALID_CLAIMS	Thông tin token không hợp lệ
INVALID_ISSUER	Nguồn phát hành token không hợp lệ
INVALID_AUDIENCE	Đối tượng token không hợp lệ
TOKEN_INVALIDATED	Token đã bị vô hiệu hóa
MISSING_AUTH_HEADER	Thiếu Authorization header
INVALID_AUTH_HEADER	Định dạng Authorization header không hợp lệ
MISSING_TOKEN	Thiếu token xác thực
NOT_AUTHENTICATED	Chưa xác thực
INVALID_AUTH_CONTEXT	Ngữ cảnh xác thực không hợp lệ
ADMIN_ACCESS_REQUIRED	Cần quyền quản trị viên
FORBIDDEN	Bạn không có quyền thực hiện hành động này
BAD_REQUEST	Yêu cầu không hợp lệ
EMAIL_NOT_VERIFIED	Email chưa được xác thực
EMAIL_ALREADY_VERIFIED	Email đã được xác thực
INVALID_OTP	Mã xác thực không đúng
OTP_EXPIRED	Mã xác thực đã hết hạn
LOGIN_METHOD_MISMATCH	Tài khoản này đã đăng ký bằng Google. Vui lòng đăng nhập bằng Google hoặc liên hệ quản trị viên để hỗ trợ.
EMAIL_NOT_REGISTERED	Email không tồn tại trong hệ thống
INVALID_USERNAME	Username phải từ 3-20 ký tự
PASSWORD_TOO_SHORT	Mật khẩu phải có ít nhất 8 ký tự
PASSWORD_TOO_WEAK	Mật khẩu phải chứa ít nhất 1 chữ hoa, 1 chữ thường, 1 số và 1 ký tự đặc biệt (@\$!*%?&)

Table 6.1: Application Error Codes

INTERNAL_ERROR	Lỗi hệ thống
NO_FIELDS_TO_UPDATE	Không có trường nào để cập nhật
INVALID_ID	Định dạng ID không hợp lệ
PAGINATION_INVALID	Số trang hoặc kích thước trang không hợp lệ. Kích thước trang phải nhỏ hơn 500.
USER_NOT_FOUND	Không tìm thấy người dùng
USERNAME_EXISTS	Tên người dùng đã tồn tại
EMAIL_EXISTS	Email đã được sử dụng
USER_INACTIVE	Tài khoản người dùng đã bị vô hiệu hóa
INVALID_GENDER	Giá trị giới tính không hợp lệ
INVALID_DATE_FORMAT	Định dạng ngày không hợp lệ, sử dụng YYYY-MM-DD
AGE_TOO_YOUNG	Phải từ 13 tuổi trở lên
INVALID_BIRTH_DATE	Ngày sinh không hợp lệ
INVALID_PROVINCE	Tỉnh/thành phố không hợp lệ
TOO_MANY_INTERESTS	Tối đa 10 sở thích
INVALID_INTEREST	Sở thích không hợp lệ
COMMUNITY_NOT_FOUND	Không tìm thấy cộng đồng
COMMUNITY_NAME_EXISTS	Tên cộng đồng đã tồn tại
COMMUNITY_DELETED	Cộng đồng đã bị xóa
MODERATOR_ALREADY_EXISTS	Người dùng đã là moderator của cộng đồng
CANNOT_REMOVE_MODERATOR	Không thể xóa moderator này
CANNOT_REMOVE_CREATOR	Không thể xóa người tạo cộng đồng khỏi danh sách moderator
BANNED_COMMUNITY	Người dùng đã bị ban khỏi cộng đồng
USER_NOT_MEMBER	Bạn chưa tham gia cộng đồng này
MEMBERSHIP_NOT_FOUND	Không tìm thấy thành viên
ALREADY_MEMBER	Bạn đã là thành viên của cộng đồng này
MEMBERSHIP_CREATE_FAILED	Không thể tạo thành viên
MEMBERSHIP_DELETE_FAILED	Không thể xóa thành viên
INVALID_MEMBERSHIP_DATA	Dữ liệu thành viên không hợp lệ

Table 6.1: Application Error Codes (Continued)

MEMBERSHIP_ALREADY_PROCESSED	Yêu cầu tham gia đã được xử lý
POST_NOT_FOUND	Không tìm thấy bài viết
VOTE_NOT_FOUND	Không tìm thấy bình chọn
POLL_ALREADY_VOTED	Bạn đã bình chọn lựa chọn này rồi
POLL_CANNOT_EDIT	Không thể chỉnh sửa bình chọn sau khi đã có người bình chọn
ALREADY_REPORTED	Bạn đã báo cáo nội dung này rồi
DRAFT_NOT_FOUND	Không tìm thấy bản nháp
COMMENT_NOT_FOUND	Không tìm thấy bình luận
DEPTH_TOO_HIGH	Độ sâu phải từ 0 đến 2
USER_MUTED	Người dùng đã bị cấm bình luận
CHANNEL_NOT_FOUND	Không tìm thấy kênh
NO_MESSAGE_FOUND	Không tìm thấy tin nhắn
REPORT_NOT_FOUND	Không tìm thấy report

Table 6.1: Application Error Codes